

Chapter 1

Contents

Chapters

1. Introduction to Computer Architecture & Organization
 - 1.1 Digital Electronics Fundamentals
 - 1.2 Functional units of a computer.
 - 1.3 Bus structures
 - 1.4 Memory Organization
 - 1.5 Von Neumann & Harvard Architectures
 - 1.6 Basic Instruction formats
 - 1.7 Operational Concept of a computer
 - 1.8 Register Bus Structure
 - 1.9 Configuration and design of units of CPU
 - 1.9.1 ALU
 - 1.9.2 Shift/Rotate Unit
 - 1.9.3 Combined ALU, Shift/Rotate & Multi bus unit.
 - 1.10 Optimization
 - 1.11 Timing & Control Unit
 - 1.12 Pipelining

1.1 Digital Electronics Fundamentals

The basic concept of Microcontrollers starts with number system and digital electronics concepts. They are presented as follows.

1.1.1 Number System

Numbers are typically classified as

- Decimal
- Octal
- Hexa-Decimal
- Binary

Decimal Numbers: The decimal numbers are cascaded digits of numbers 0 to 9. So, the digits possible are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

Octal Numbers: The decimal numbers are cascaded digits of numbers 0 to 7. So, the digits possible are 0, 1, 2, 3, 4, 5, 6, 7.

Hexa-Decimal Numbers: The decimal numbers are cascaded digits of numbers 0 to 9 along with character A, B, C, D, E, F representing 10, 11, 12, 13, 14 and 15. So the digits possible are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.

Binary Numbers: The decimal numbers are cascaded digits of 0's and 1's.

1's Complement & 2's Complement:

The 2's complement of a number is the 1's complement of the given number + 1. The 1's complement of a number is obtained by subtracting the given number from a set of number with 1's.

Table 1.1: 1's and 2's Complement numbers for 0 to 7 decimal numbers.

Decimal	Binary	1's Complement	2's Complement
0	000	1111 (or 0000)	0000
1	001	1110	1111
2	010	1101	1110
3	011	1100	1101
4	100	1011	1100
5	101	1010	1011
6	110	1001	1010
7	111	1000	1001

	1	1		1	1		1	1		1	1			
0	0	0		1	0	1		2	1	0		3	1	1
1's	1	1		1	0			0	1			0	0	
		+1			+1				+1				+1	
2's	0	0		1	1			1	0			0	1	

9's Complement & 10's Complement:

The 10's complement of a number is the 9's complement of the given number + 1. The 9's complement of a number is obtained by subtracting the given number from a set of number with 9's.

9's complement of a number 1234

$$\begin{array}{r} 9999 \\ - 1234 \\ \hline 8765 \end{array}$$

10's complement of a number 1234

$$\begin{array}{r} 9999 \\ - 1234 \\ \hline 8765 \\ + \quad 1 \\ \hline 8766 \end{array}$$

$$\begin{array}{r} 9876 \Rightarrow 9876 \\ - 1234 \quad +8765 \text{ (9's)} \\ \hline 8642 \quad 1 \ 8641 \\ \quad + \quad 1 \\ \hline 8642 \end{array}$$

$$\begin{array}{r} 1234 \Rightarrow 1234 \\ - 9876 \quad + 0123 \text{ (9's)} \\ \hline 8642 \quad 1357 \\ \quad + 8642 \text{ (9's)} \\ \hline -8642 \end{array}$$

The 10's Complement subtraction is done by getting the 10's complement of the subtrahend and adding it to the minuend. A carry result indicates a positive number and can be neglected. If there is no carry indicates result is negative and is a 10's complement number. The 10's complement of this negative result number may be taken and assigned with a minus sign for a proper number.

With Carry

$$\begin{array}{r} 9876 \Rightarrow 9876 \\ - 1234 \quad +8765 \text{ (9's)} \\ \hline 8642 \quad + \quad 1 \text{ (10's)} \\ \hline 1 \ 8642 \\ \quad + 8641 \text{ (9's)} \\ \hline 8642 \end{array}$$

Without Carry

$$\begin{array}{r} 1234 \Rightarrow 1234 \\ - 9876 \quad + 0123 \text{ (9's)} \\ \hline 8642 \quad + \quad 1 \text{ (10's)} \\ \hline 1358 \\ \hline -8642 \end{array}$$

Unsigned Numbers: Unsigned numbers are numbers that have no sign bit and are thus positive numbers.

Table 1.2: Signed Magnitude numbers for 0 to 7 decimal numbers.

Number	Signed Magnitude	Number	Signed Magnitude
0	0000	0	1000
1	0001	-1	1001
2	0010	-2	1010
3	0011	-3	1011
4	0100	-4	1100
5	0101	-5	1101
6	0110	-6	1110
7	0111	-7	1111

Signed 2's complement numbers: In a signed 2's complement number the MSB is sign bit indicating a positive or negative. The rest of the bits represents 2's complement of a given number.

Table 1.3: Signed 2's Complement numbers for 0 to 7 decimal numbers.

Number	Signed 2's Complement	Number	Signed 2's Complement
0	0000	0	0000
1	0001	-1	1111
2	0010	-2	1110
3	0011	-3	1101
4	0100	-4	1100
5	0101	-5	1011
6	0110	-6	1010
7	0111	-7	1001

Signed Numbers: Signed numbers are numbers make use of a sign bit that differentiate whether a number is positive or negative. The most significant bit: MSB in a number is called the sign bit. A negative number is represented by a sign bit of 1 and a positive number is represented by a sign bit of 0. Signed numbers could be a signed magnitude numbers or a signed 2's complement numbers.

Signed magnitude numbers: In a signed magnitude number the MSB is sign bit indicating a positive or negative. The rest of the bits represents the magnitude of the number.

Complement examples

1	0001	2	0010	3	0011	4	0100
-1 (1's)	1110	-2 (1's)	1101	-3 (1's)	1100	-4 (1's)	1011
+1	+1	+1	+1	+1	+1	+1	+1
-1 (2's)	1111	-2 (2's)	1110	-3 (2's)	1101	-4 (2's)	1100
5	0101	6	0110	7	0111		
-5 (1's)	1010	-6 (1's)	1001	-(1's)	1000		
+1	+1	+1	+1	+1	+1		
-5 (2's)	1011	-6 (2's)	1010	-7 (2's)	1001		

Binary Addition Examples using 2's complement

Addition between two positive number of two negative numbers

</




-2 (2's)	1	1	1	0	2's Complement Addition
-3 (2's)	1	1	0	1	2's Complement Addition
Result = -5	1	0	1	1	-ve

Overflow occurring additions resulting in wrong results.

</



-5 (2's)	1	0	1	1	2's Complement Addition
-4 (2's)	1	1	0	0	2's Complement Addition
Result = -9	0	1	1	1	+ve

Binary Subtraction Examples - 2's Complement:

Addition between a positive number and a negative number.

Binary subtraction is done by taking 2's Complement of the subtrahend and adding it to the minuend. The sign bit indicates whether the result is positive or negative. If there is a carry out, ignore it and if MSB is 0 the result is positive and in proper form. If MSB is 1 irrespective of carry it indicates result is negative and is in 2's complement form.

5				0	1	0	1
-3 (3's)	1	1	0	1	2's Complement Addition		
Result = 2	0	0	1	0	+ve		

					
3	0	0	1	1	
-2 (2's)	1	1	1	0	2's Complement Addition
Result = 1	0	0	0	1	+ve

2	0	0	1	0	
-3 (3's)	1	1	0	1	2's Complement Addition
Result = -1	1	1	1	1	-ve

						
-3 (2's)	1	1	0	1	2's Complement	
-2 (2's)	1	1	1	0	2's Complement Addition	
Result = -5	1	0	1	1	+ve	

It must be determined if an Overflow has occurred or not for which Carry into and out of sign bit must be considered to indicate the positive or a negative number. Overflow concept is discussed later in the operations on numbers and flags sections.

1.1.2 Logic Gates

The digital circuits are classified as combinational and sequential circuits. Any digital circuit is composed of basic gates such as buffer, inverter, AND, OR XOR, XNOR, and universal gates – NAND & NOR.

- Buffer

A “BUFFER” is a basic gate that transmits the data signal that is available at its input to the output. It is generally used as a driver to maintain the required logic levels and for driving the bus loads. The three state buffers are a type of buffers which maintain a high impedance (Hi-Z) state when the device is disabled. The truth table of Buffer and Tri-State Buffer is presented in Table 1.4.

- NOT

A “NOT” or “INVERTER” is a gate that transmits the inverse (Not) of data signal that is available at its input to the output. It is generally used as a driver to maintain the required inverse logic levels, for driving the bus loads. The three state inverters are a type of Not or inverters which maintain a high impedance (Hi-Z) state when the device is disabled. The truth table of NOT Gate or Inverter is presented in Table 1.4.

- AND

An “AND” gate is a gate that performs logical AND operation on its inputs for providing the output signal. The number of inputs may vary, i.e. 2 Input, 3 input etc. The truth table of AND Gate is presented in Table 1.4.

- OR

An “OR” gate is a gate that performs logical OR operation on its inputs for providing the output signal. The number of inputs may vary, i.e. 2 Input, 3 input etc. The truth table of OR Gate is presented in Table 1.4.

- XOR

An “XOR” gate is a gate that performs logical XOR operation on its inputs for providing the output signal. The number of inputs may vary, i.e. 2 Input, 3 input etc. The truth table of XOR Gate is presented in Table 1.4.

- XNOR

An “XNOR” gate is a gate that performs logical XNOR operation on its inputs for providing the output signal. The number of inputs may vary, i.e. 2 Input, 3 input etc. The truth table of XNOR Gate is presented in Table 1.4.

Table 1.4: Truth Tables for different Logic Gates.

Buffer		
En	I/P	O/P
1	0	0
1	1	1
0	x	0

Inverter (NOT)		
En	I/P	O/P
1	0	1
1	1	0
0	x	0

Tri-State Buffer		
En	I/P	O/P
1	0	0
1	1	1
0	x	Z

Tri-State Inverter		
En	I/P	O/P
1	0	1
1	1	0
0	x	Z

AND 2-I/P		
A	B	O/P
0	0	0
0	1	0
1	0	0
1	1	1

OR 2-I/P		
A	B	O/P
0	0	0
0	1	1
1	0	1
1	1	1

XOR 2-I/P		
A	B	O/P
0	0	0
0	1	1
1	0	1
1	1	0

XNOR 2-I/P		
A	B	O/P
0	0	1
0	1	0
1	0	0
1	1	1

NAND 2-I/P		
A	B	O/P
0	0	1
0	1	1
1	0	1
1	1	0

NOR 2-I/P		
A	B	O/P
0	0	1
0	1	0
1	0	0
1	1	0

Here En stands for Enable, I/P stands for input and O/P stand for output.

- **NAND**

A “NAND” gate is a gate that performs logical AND operation on its inputs and provides the output signal which is an inversion of AND operation. It is one of the two universal gates. The number of inputs may vary, i.e. 2 Input, 3 input etc. The truth table of NAND Gate is presented in Table 1.4.

- **NOR**

A “NOR” gate is a gate that performs logical OR operation on its inputs and provides the output signal which is an inversion of OR operation. It is one of the two universal gates. The number of inputs may vary, i.e. 2 Input, 3 input etc. The truth table of NOR Gate is presented in Table 1.4.

1.1.3 Operations on Numbers & Flags

Various operations are carried out on binary numbers in digital computer/computing devices. Some of the arithmetic operations are Addition, Subtraction and logical operations are AND, OR, XOR etc. After these operations are performed in arithmetic and logic unit - ALU, the processor updates some flags to indicate certain important information. This information is often required to determine the next sequence of operations that must be performed.

Flags are status bits which indicate the status after an operation in an ALU. Flags are of different types. The most common flags are Carry Flag, Auxiliary Carry Flag, Zero Flag, Parity Flag, Sign Flag, Overflow Flag. The operations on binary numbers and corresponding status flag update is presented here for different operations.

Carry Flag:

The carry flag is used to indicate whether the result exceeded the size of the given register. As an example, if an 8-bit ALU operation is performed, if it results in a 9-bit result due to an addition or a borrow in a subtraction the Carry Flag is generally set. In the common bit notation following 0 to 7 ordering for an 8-bit number, if there is a ‘1’ into the 8th bit due to operation on the 7th bit it is considered as a carry. Similarly, a borrow from 8th bit to 7th bit in a subtraction is considered as a carry. The carry is not affected for most logical operations.

Addition operation

								
7654	3210				7654	3210		
0001	0010	(12H)			1001	0010	(92H)	
+ 0001	0011	(13H)			+ 1001	0011	(93H)	
0 0010	0101	(0 25H)	CF → 0		1 0010	0101	(1 25H)	CF → 1

Subtraction operation

0001	0011	(13H)		1001	0010	(92H)	
- 0001	0010	(12H)		- 1001	0011	(93H)	
0	0000	0001	(0 01H)	1	1111	1111	(1 FFH)
			CF → 0				= - 01H

Auxiliary Carry Flag

The auxiliary carry flag is used in BCD operations. It is set when carry occurs from lower digit to the next higher digit. In the common bit notation following 0 to 7 ordering for an 8-bit number, if there is a '1' into the 4th bit due to operation on the 3rd bit it is considered as an auxiliary carry to indicate a carry from 3rd bit to the 4th bit. The auxiliary carry is not affected for logical operations.

Addition operation

0001	0010	(12H)		0001	1010	(1AH)	
+ 0001	0011	(13H)		+ 0001	1011	(1BH)	
0010	0101	(25H)	ACF → 0	0011	0101	(35H)	ACF → 1

Zero Flag

The zero flag is used to indicate whether the result of an operation is a zero value. It is set to 1 when the result is zero and reset to 0 when the result is non zero. In the common bit notation following 0 to 7 ordering for an 8-bit number, if there are all 0's in the result the ZF is set to 1, otherwise it is set to a 0. The zero flag is generally affected for logical operations.

0001	0010	(12H)		1111	1111	(FFH)	
+ 0001	0011	(13H)		+ 0000	0001	(01H)	
0010	0101	(25H)	ZF → 0	1	0000	0000	(1 00H)
							ZF → 1

Parity Flag

The parity flag is used to indicate whether the result of an operation resulted in an even number of 1's. In the common bit notation following 0 to 7 ordering for an 8-bit number, PF is set to 1 when the number of 1's in the result is even and reset to 0 when the number of 1's in the result is 0. The parity flag is generally affected for logical operations.

0001	0010	(12H)		0001	0001	(11H)	
+ 0001	0011	(13H)		+ 0111	0111	(77H)	
0010	0101	(25H)	PF → 0	1000	1000	(88H)	PF → 1

Sign Flag

The sign flag is used to indicate whether the result of an operation is a negative value. In the common bit notation following 0 to 7 ordering for an 8-bit number, SF is set to 1 when the 7th bit of the result is 1 and reset to 0 when the 7th bit of result is a 0. The sign flag is generally affected for logical operations.

0001 0010 (12H)	0000 0001 (01H)
+ 0001 0011 (13H)	+ 1111 1110 (FEH) = - 02H
0 010 0101 (25H) SF → 0	1 111 1111 (FFH) = - 01H SF → 1

Overflow Flag

The overflow flag is used for signed operations to indicate whether the result of an operation exceeded the given bit value, modified the sign bit or if the result is not the correct value. In the common bit notation following 0 to 7 ordering for an 8-bit number, OF is set to 1 when a 1 is generated into the 8th bit but not into 7th bit or vice versa. The OF is reset to 0 when a 1 is generated into both the 8th bit and 7th bit or when a 1 is not generated into both the bits. The overflow flag is generally not affected for logical operations.

Addition operation

7654 3210	7654 3210
0 into both 7 th and 8 th bits	0 into 7 th and 1 into 8 th bit
0001 0010 (12H)	1001 0010 (92H)
+ 0001 0011 (13H)	+ 1001 0011 (93H)
0 0010 0101 (0 25H) OF → 0	1 0010 0101 (1 25H) OF → 1

1 into both 7 th and 8 th bits	1 into 7 th and 0 into 8 th bit
1100 0010 (C2H)	0101 0010 (52H)
- 1101 0011 (D3H)	- 0101 0011 (53H)
1 1001 0101 (1 95H) OF → 0	0 1010 0101 (0 A5H) OF → 1

Examples for Addition operation and corresponding flags

Addition of two 8-Bit numbers

0001 0001	1011 1101	1101 1101	1111 0001
+ 0001 0001	+ 1011 1101	+ 1101 1101	+ 0000 1111
0 0010 0010	1 0111 1010	1 1011 1010	1 0000 0000
CF = 0	CF = 1	CF = 1	CF = 1
AC = 0	AC = 1	AF = 1	AF = 1
ZF = 0	ZF = 0	ZF = 0	ZF = 1
PF = 1	PF = 0	PF = 0	PF = 1
SF = 0	SF = 0	SF = 1	SF = 0
OF = 0	OF = 1	OF = 0	OF = 0

Examples for other operations and corresponding flags

Subtraction	XOR	AND	OR
0101 0010 (52H)	1011 1101	1101 1101	1110 0101
- 0111 0101 (75H)	<u>1010 1010</u>	<u>0101 1010</u>	<u>0101 0111</u>
1 1101 1101 (1 DDH)	0001 0111	0101 1000	1111 0111

CF → 1	CF = Unaffected	CF = Unaffected	CF = Unaffected
AC = 1	AC = Unaffected	AF = Unaffected	AF = Unaffected
ZF = 0	ZF = 0	ZF = 0	ZF = 0
PF = 1	PF = 1	PF = 0	PF = 0
SF = 0	SF = 0	SF = 0	SF = 1
OF = 0	OF = Unaffected	OF = Unaffected	OF = Unaffected

1.1.4 Combinational Circuits

The digital circuits are a combination of basic gates primarily categorized as combinational and sequential circuits. Combinational circuits are the form of circuits where the next output is independent of previous inputs or outputs. The most common combinational circuits relevant for processors and controllers are

- Half/Full Adder
- Half/Full Subtractor
- Ripple Carry Adder
- Encoder
- Decoder
- Multiplexer

Microprocessors and Microcontrollers perform several arithmetic and logical operations. These operations are performed by the ALU – Arithmetic & Logic Unit. Some of these operations are addition, subtraction, multiplication and division. The most common part of these operations is addition which is performed by adders - Half Adders, Full Adders. Subtractors as such are not used, as subtraction is normally performed through addition of 2's complement numbers. The truth table for Half Adder, Half Subtractor, Full Adder and Full Subtractor are shown in Table 1.5. The combinational circuits for 1-bit operations are shown in Fig 1.1 and Fig 1.2. The ALUs typically perform operation on group of bits from 8 to 64-bits. A typical Full Adder structure that can perform 4-bit operations – Ripple Carry Adder is shown in Fig 1.3.

Table 1.5: Truth Tables for Adders and subtractors

Half Adder			
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Half Subtractor			
A	B	Diff	Borrow
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Full Adder				
A	B	C	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full Subtractor				
A	B	C	Diff	Borrow
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

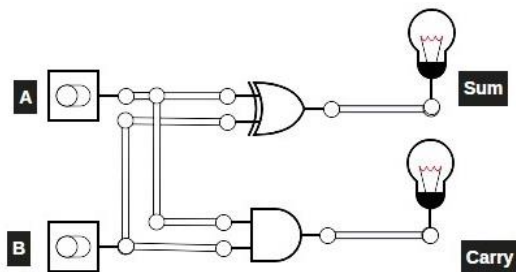


Fig 1.1a: Half Adder

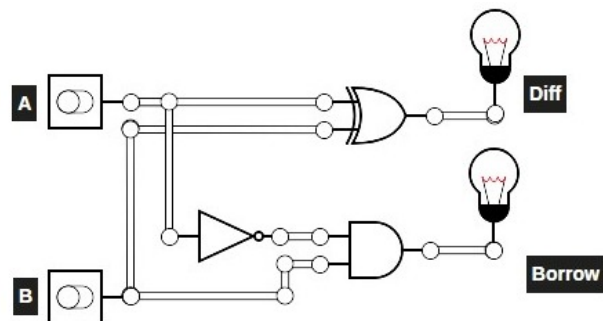


Fig 1.1b: Half Subtractor

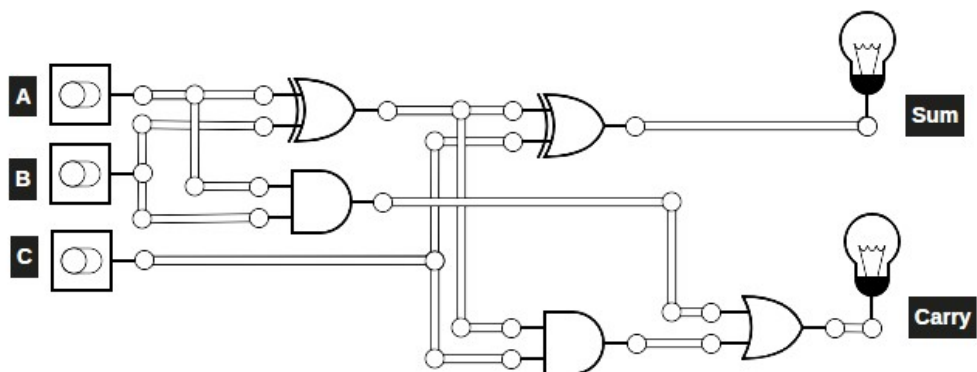


Fig 1.2a: Full Adder

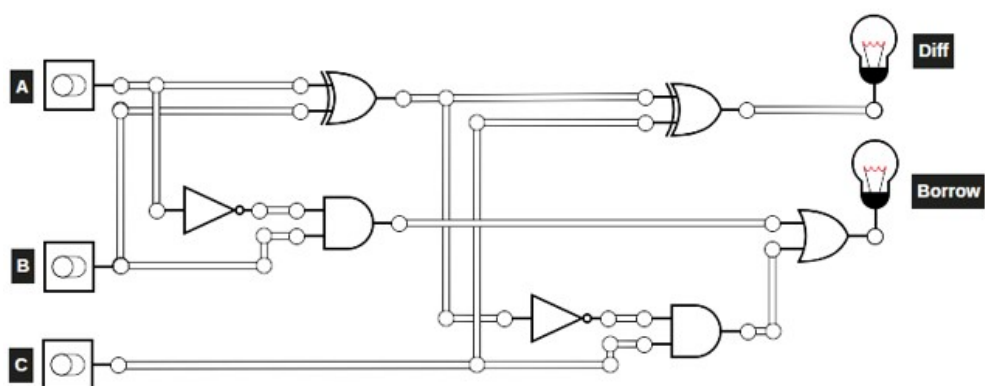


Fig 1.2b: Full Subtractor

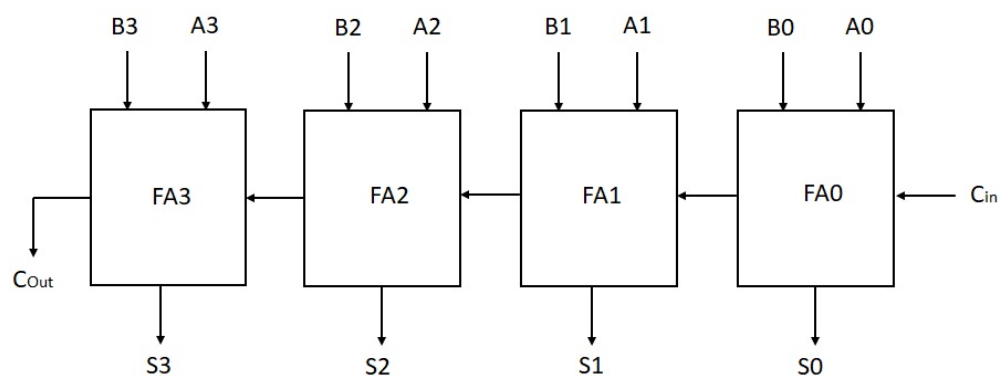


Fig 1.3: Ripple Carry Adder

The Half Adder Sum and Carry expressions are

$$H_k = A_k \oplus B_k = A_k \bar{B}_k + \bar{A}_k B_k \quad (1.1)$$

$$C_k = A_k \cdot B_k \quad (1.2)$$

Here k represents the number of the stage.

The Full Adder Sum and Carry expressions are

$$\text{Sum} = H_k \oplus C_{k-1} = C_{k-1} \bar{H}_k + \bar{C}_{k-1} H_k \quad (1.3)$$

$$\text{Carry} = A_k \cdot B_k + H_k \cdot C_{k-1} \quad (1.4)$$

The Encoders and Decoders are integral part of the operations of Microprocessors and Microcontrollers. The instructions used to perform specific operations are encoded in the form of an instruction code and the internal control circuitry is configured based on this encode value. The instruction encoding and design mechanism is presented in ALU design section 1.9. The truth table for a 3-bit: 8x3 encoder is shown in Table 1.6 and equivalent circuit is shown in Fig 1.4.

A control unit exists in the CPU which decodes the instruction code in the process of configuring the internal circuitry. It is done in a sequence of steps based in a specific order and is timed based on the clock cycles. Hence it is called the timing and control unit. This process makes use of a decoder to generate some control signals based on the decoded value. The truth table for a 3-bit: 3x8 decoder is shown in Table 1.7 and equivalent circuit is shown in Fig 1.5. The instruction decoding and timing and control design mechanism is presented in ALU design section 1.11.

Table 1.6: Encoder

$I_7 I_6 I_5 I_4 I_3 I_2 I_1 I_0$	$E_2 E_1 E_0$
0 0 0 0 0 0 0 1	0 0 0
0 0 0 0 0 0 1 0	0 0 1
0 0 0 0 0 1 0 0	0 1 0
0 0 0 0 1 0 0 0	0 1 1
0 0 0 1 0 0 0 0	1 0 0
0 0 1 0 0 0 0 0	1 0 1
0 1 0 0 0 0 0 0	1 1 0
1 0 0 0 0 0 0 0	1 1 1

$$E_2 = I_7 + I_6 + I_5 + I_4.$$

$$E_1 = I_7 + I_6 + I_3 + I_2.$$

$$E_0 = I_7 + I_5 + I_3 + I_1.$$

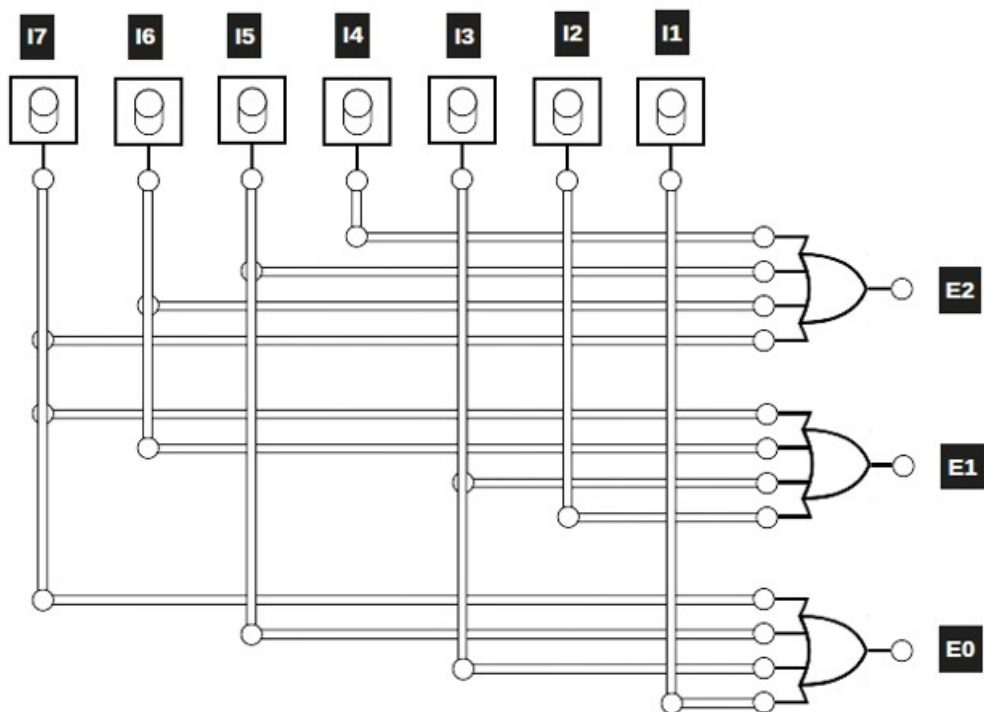


Fig 1.4: Encoder 8x3

Table 1.7: Decoder

I_2	I_1	I_0	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

$$D_0 = I_2' \cdot I_1' \cdot I_0'$$

$$D_1 = I_2' \cdot I_1' \cdot I_0$$

$$D_2 = I_2' \cdot I_1 \cdot I_0'$$

$$D_3 = I_2' \cdot I_1 \cdot I_0$$

$$D_4 = I_2 \cdot I_1' \cdot I_0'$$

$$D_5 = I_2 \cdot I_1' \cdot I_0$$

$$D_6 = I_2 \cdot I_1 \cdot I_0'$$

$$D_7 = I_2 \cdot I_1 \cdot I_0$$

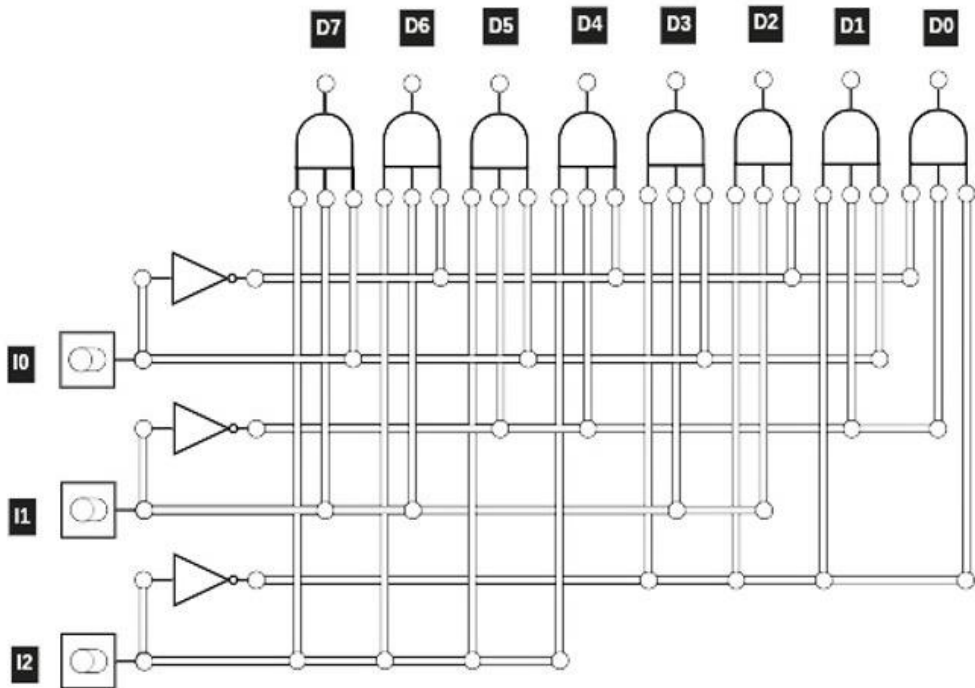


Fig 1.5: Decoder 3x8

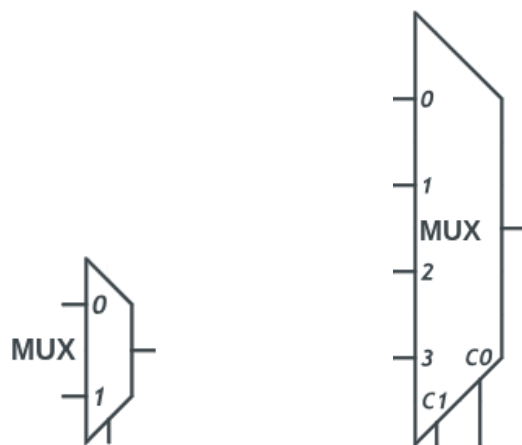


Fig 1.6: Multiplexer 2:1 and 4:1

Another common combinational circuit that is commonly used is the multiplexer. A specific signal line may be selected from a group of signal lines may be selected and passed on to the next stage of circuits using multiplexers. A circuit to perform Shift or Rotate operations using multiplexers is presented in Section 1.9. Here multiplexer inputs are connected in such a way that the select lines of a multiplexer determine the operation that

is performed in a shift register. The truth table for a 2:1 & 4:1 multiplexer is shown in Table 1.8 and general representation of multiplexers is shown in Fig 1.6.

Table 1.8: Multiplexer 2:1 and 4:1

C_0	O/P
0	I_0
1	I_1

C_1	C_0	O/P
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Here C_1 and C_0 are select lines and I_3, I_2, I_1 and I_0 are input lines.

For 2:1 multiplexer O/P $Z = C_0' \cdot I_0 + C_0 \cdot I_1$

For 4:1 multiplexer O/P $Z = C_1' \cdot C_0' \cdot I_0 + C_1' \cdot C_0 \cdot I_1 + C_1 \cdot C_0' \cdot I_2 + C_1 \cdot C_0 \cdot I_3$

1.1.5 Sequential Circuits

Sequential circuits are form of circuits where the next output is dependent of previous inputs or outputs. Such circuits generally include a memory element. The most common sequential circuits relevant for processors and controllers are

- Latches/Flip Flops
- Registers (Shift)
- Counters – Synchronous/Asynchronous

Latches/Flip Flops:

Flip Flops are basic sequential circuits and digital storage elements capable of storing information. They are operated through a clock, and the output is thus affected based on the input only at specific instance like a rising or a falling edge of a clock cycle. Latches on the hand produces a change in output whenever input changes and are generally level triggered.

Table 1.9: Flip-Flops

R-S			
Clk	R	S	Q_{n+1}
0	x	x	Q_n
1	0	0	NC
1	0	1	0
1	1	0	1
1	1	1	UD

J-K			
Clk	J	K	Q_{n+1}
0	x	x	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	Q_n'

Here

NC is No change; UD is Undefined or forbidden state.

Q_n is present state and Q_{n+1} is next state.

T		
T	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	0

D		
D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	0
1	1	1

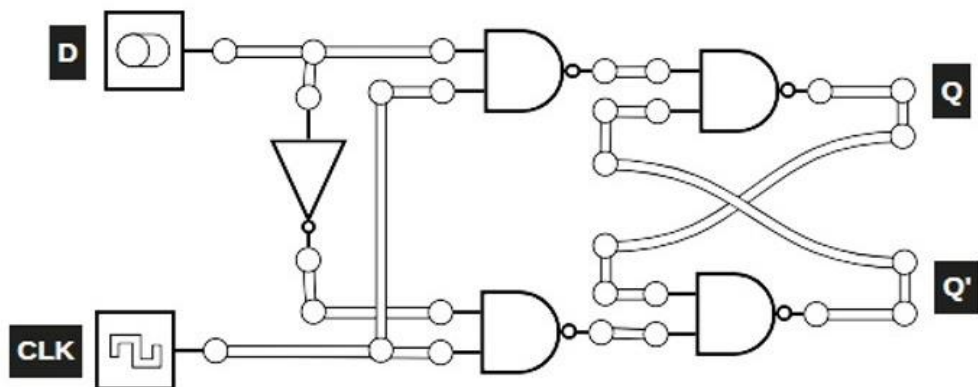


Fig 1.7: Flip Flop

The four main types are R-S, J-K, T and D Flip Flops/Latches. The most used is the D-Flip Flop which are used as data storage elements like general purpose or special purpose registers or shift registers etc. Latches are typically used in the Port I/O Pin structures to produce an immediate change in output based on its inputs to allow for RC delay effects. The typical structure of a D Flip Flop is shown in Fig 1.7 and truth tables for different flip flops is shown in Table 1.9.

A microprocessor or microcontroller typical has several electronic components that typically are activated using a clock or enable signal. Latches employ enable signal instead of a clock signal.

Registers (Shift):

Registers are sequential circuits consisting of a group of flip flops typically with size of 8, 16, 32 or 64 bits used to store digital information having a combination of 1's and 0's. These are made of D flip flops and used for general purpose or special purpose register storage in microprocessors or microcontrollers. These are typically parallel in parallel out shift registers. A typical parallel in parallel out register is shown in Fig 1.8a.

The Microprocessors or Microcontrollers perform operations such as addition, subtraction, multiplication, shifting etc. on group of bits using ALU – Arithmetic Logic Unit. Hence it uses registers for temporary storage of data inside its CPU. Microcontrollers typically employ serial communication components and protocols such as USART for serial communication over RS232, RS485 or other serial bus such as I2C, SPI or USB. Hence a conversion from serial to parallel and vice versa is required. So, these devices make use of serial to parallel conversion devices such as serial to parallel shift registers or parallel to serial shift registers. Variations of shift registers such as serial in serial out, serial in parallel out, parallel in serial out shift registers exists that are used for serial to parallel data transfer and vice versa.

For serial-to-serial communication the serial in serial out registers are used. Whenever data is to be sent serially over a serial communication line, data from a register consisting of several bits is copied into a parallel in serial out register. On the other end of the serial line, a serial in parallel out register is used to receive data serially and the data is made available parallel to be stored in serial input register. The typical structure of serial in serial out shift register is shown in Fig 1.8b and a structure of serial in parallel out shift register is shown in Fig 1.8c. A single structure that can perform any type of shift or rotate operations is presented in later section of this chapter that shows the design of Logic shift rotate unit in Sec 1.9.

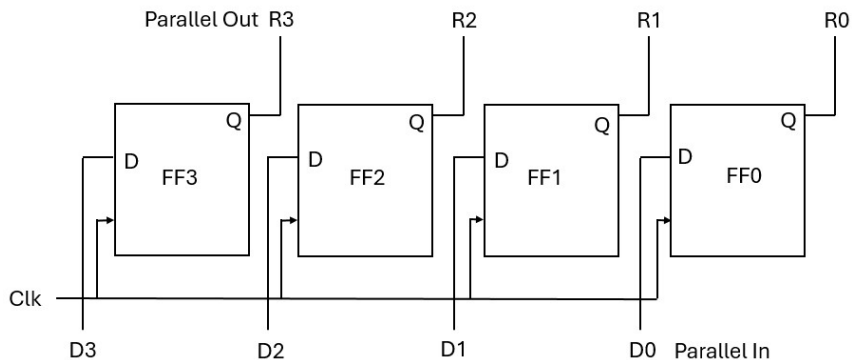


Fig 1.8a: Register – Parallel In Parallel Out

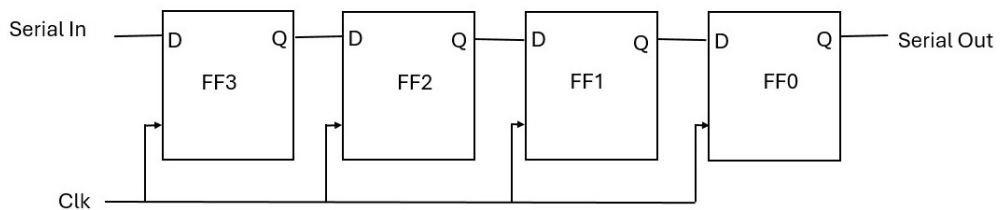


Fig 1.8b: Register – Serial In Serial Out

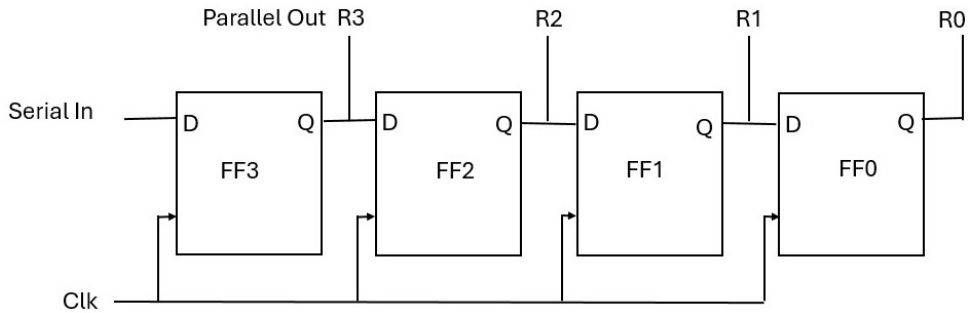


Fig 1.8c: Register – Serial In Parallel Out

Counters:

Counter is another sequential circuit that counts or keeps track of occurrence of a certain number of events. In functionality Timers and counter are similar, while the timers keep track of number of input clock pulses, a counter keeps track of events which can be periodic or aperiodic. Based on the design, the circuit can increment or decrement the count. Counters are two types. They are

- a) Asynchronous Counters
- b) Synchronous Counters.

Asynchronous vs Synchronous Counters

The asynchronous and synchronous counters vary mainly with respect to clock. Asynchronous counters as the name suggest makes use of flip flops that uses a signal whose input clock period may vary from one flip flop to another as output of one flip flop drives the next flip flop. Only the first flip flop may use the main clock signal. The Synchronous counter on the other hand uses a universal clock signal for all the flip flops are simultaneously operated and all the outputs are updated synchronously or parallel and is faster than asynchronous counters as it eliminates cumulative stage delays. A basic 4-bit asynchronous counter is shown in Fig 1.9a and synchronous counter is shown in Fig 1.9b which increments count from 0 to 15 (F).

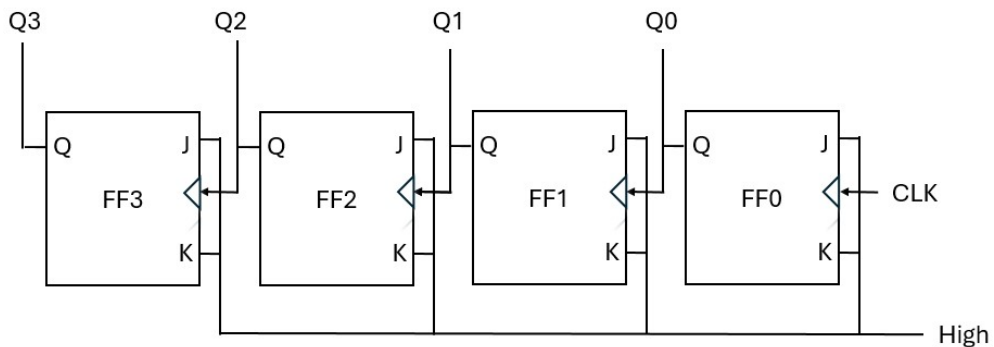


Fig 1.9a: Asynchronous Counter

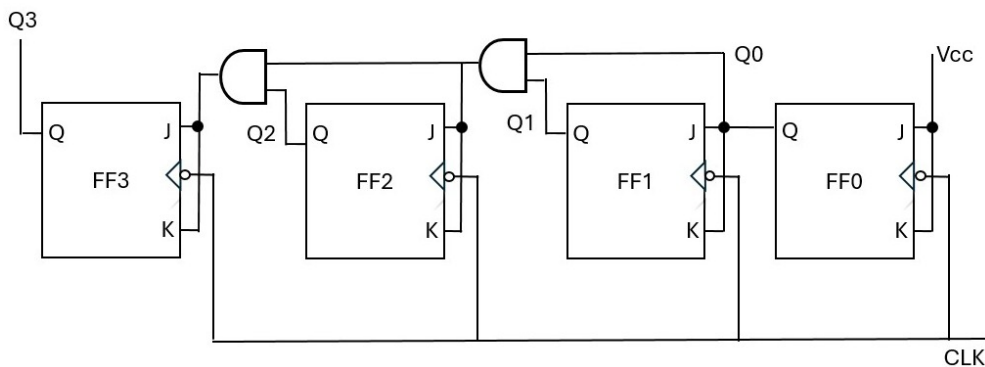


Fig 1.9b: Synchronous Counter

Microprocessors typically employ timers like watchdog timers whereas counter must be interfaced externally. A microcontroller typically employs both timers and counters that can be used as frequency dividers, generate waveforms, Pulse width modulated signals and can be used for measurements such as time, frequency, speed etc. Microcontrollers typically employ synchronous counters as they operate based on a clock.

1.2 Functional units of a computer.

A computer is a general-purpose device that is programmable. Computers are programmed to perform a set of arithmetic or logical operations. The set of arithmetic or logical operations varies based on the required application. These operations are stored as a sequence of instructions in memory and are executed serially in the Central Processing Unit - CPU by configuring the Arithmetic and Logic Unit - ALU. Since a computer is reprogrammable a different application may be executed on the computer by just altering or loading the required sequence instructions into the memory. The basic functional unit of a computer is shown in fig 1.10.

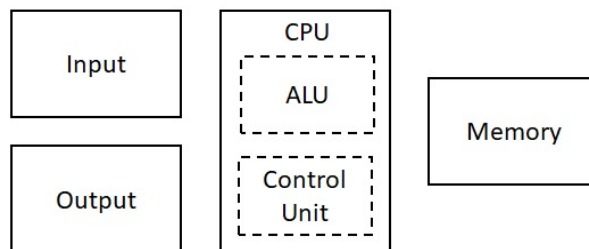


Fig 1.10: Functional units of a computer

They basic functional units of a computer are

- Input
- Output
- Memory

- CPU: Central Processing Unit

An input is a device that can provide data to the computer. Example for Input devices are Mouse, Keyboard, Scanners, Sensors etc. An output is a device that can accept data generated by the computer. Example for Output devices are Printers, Display Monitors, Actuators etc.

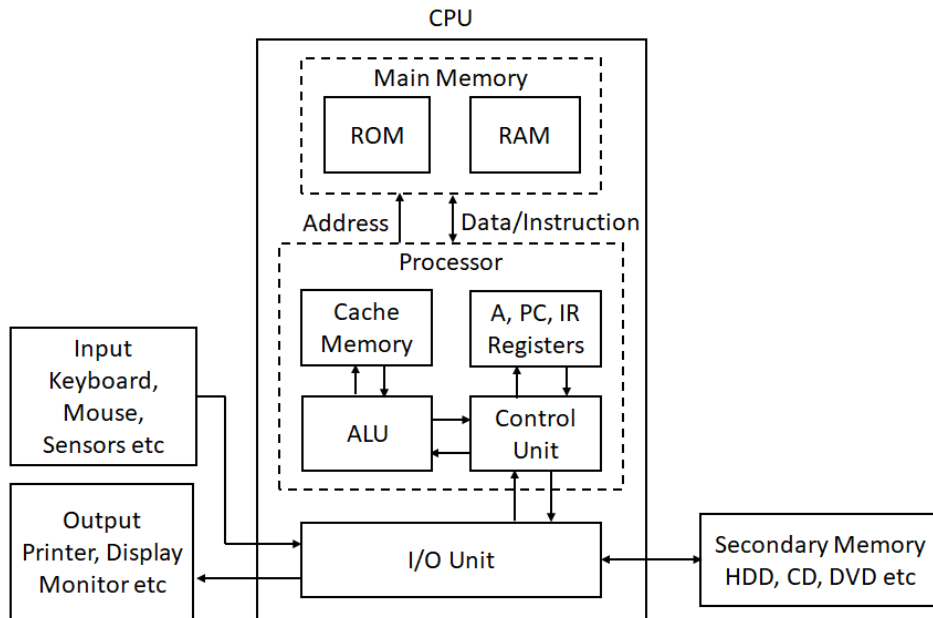


Fig 1.11: Typical computer with interconnection of functional units

Memory is the main storage block of the computer where programs or data is stored. Main memories are typically two types: RAM – Read & Write Memory, ROM – Read Only Memory. A RAM is a type of memory which can be read or written. A ROM is a type of memory which is only read.

The Central Processing Unit – CPU is the brain of the computer. It contains a control unit and an Arithmetic Logic Unit – ALU. The control unit is responsible for execution of application program instructions and ensures correct timed operation sequence. The ALU is configured by the control unit to perform arithmetic operations such as addition, subtraction, multiplication, division and logical operations such as AND, OR, NOT, shift rotate etc.

Variations of computers exist, and similarly variations exist in the number and type of internal components and functional units. A generalised computer with interconnection of different functional units is shown in fig 1.11. The figure shown in fig 1.11 consists of some addition components w.r.t the one shown in fig 1.10. The generalized structure additionally

contains units such as cache memory, registers, I/O unit, secondary memory and directional bus interconnections.

The processor typically has a cache memory and registers in addition to ALU and control unit. A register is a group of flip flops typically 8-bit, 16-bit or 24-bits. The purpose of registers is to store the information temporarily inside the processor for performing arithmetic or logical operations through the ALU. A processor typically has general purpose registers named as A, B, C, D or R0, R1, R2 etc. and special function registers such as Program Counter - PC, Instruction Register - IR, Stack Pointer - SP.

The cache memory allows for having block of code or data close to the actual processor and thus provides information at higher speeds compared to main memory. The size of cache memory is generally small; however, it operates at higher speeds compared to main memory. Though the size of main memory is larger compared to cache memory, it is however limited to tens or a few hundred blocks. Hence certain computers make use of secondary memory like HDD, SSD, CD/DVD etc. for additional information storage.

One of the other functional unit is the I/O unit which is used to interface the processor to external functional units. The input, output, external memory or other device are generally interfaced to the processor through an I/O interface unit, i.e. Input/output unit. The directional bus interconnection structure is discussed in detail in the next section.

1.3 Bus structure

A bus is a group of lines that are used for transfer of information such as address, data or control signals. They are correspondingly termed as address bus, data bus and control/status bus. Each line or a bus is either unidirectional or bidirectional.

Every endpoint (location in a memory or component, subcomponent/register etc. in a device) is associated with a unique address. They are enabled or disabled (controlled) for transfer of data from one to other using control signals.

Address Bus: The bus that transfers the value of address (device or memory) on the group of lines is termed as address bus. The size of address bus varies based on the size of the memory. It is typically 16 to 32bit. The address bus is unidirectional as the address is generated by CPU/Master.

Data Bus: The bus that transfers data on the group of lines is termed as data bus. The size of the data bus depends on the computing device. It is typically 8 to 32-bit. The data bus is bidirectional, and data may be transferred to and from the CPU/Master.

Control/Status Bus: The group of signal lines that are used to control (enable or disable) a device, memory etc. is termed as control bus. Examples for control signals are Read: RD, Write: WR, I/O or Memory: IO/M, INTR etc. In addition to control signals, certain signals that are generated are used to indicate the status of a device. This set of lines is termed as status bus. Examples for status signals are HLDA, QS1, QS0 etc.

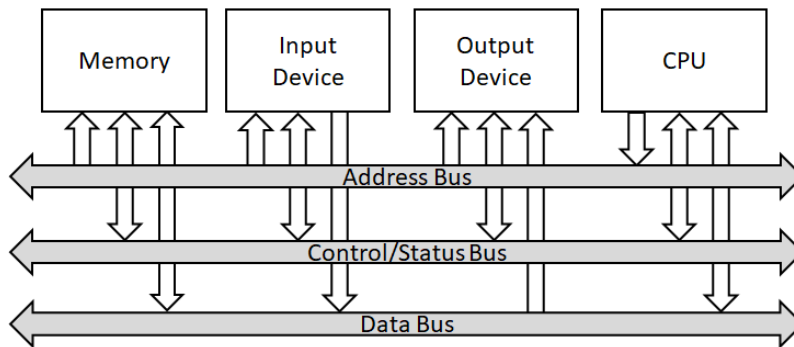


Fig 1.12: Typical Bus interconnection structure

The control/status bus are different sets of lines used for different purposes and hence may be unidirectional or bidirectional carrying control and status signals to or from the CPU/Master. The size of the control/status bus varies based on the type of package, pin configuration and peripherals of the device. A typical bus interconnection is shown in fig 1.2. It shows a bidirectional control/status bus with a unidirectional address bus.

The data bus varies based on the type of the device. A memory can be read or written and hence has bidirectional data bus. RAM memory is a read and write memory and hence has bidirectional bus. ROM memory is a read only device and is similar to an input device. An input device always provide data and hence the direction is towards the data bus for transfer of data to the CPU. An output device always accepts data and hence the direction is toward the output device for transfer of data from CPU. If the device is an I/O device (not shown in figure) where the device may function as an input or output device, the data bus is bidirectional and hence it is similar to the one shown for memory.

1.4 Memory Organization

All the computing devices consist of memory blocks to store the application program and user data. The computing devices may also organize the memory in different levels based on size, performance, speed etc. A classification of memory is presented below in Fig 1.13.

Primary Memory or Main Memory

In a computer the primary or main memory is the working memory that holds data and instructions of the application program. The RAM/ROM are considered primary memory where RAM is used for storing data and ROM is used for storing instructions of the application program. RAMs can further be classified based on the electrical structure and operating principle such as Static RAM or Dynamic RAM and sub classifications of DRAM exists and newer variations are introduced with improvement in technology. Similarly, ROM technology has evolved, and several advancements and variations are made over time such as EPROM, EEPROM & Flash memory etc.

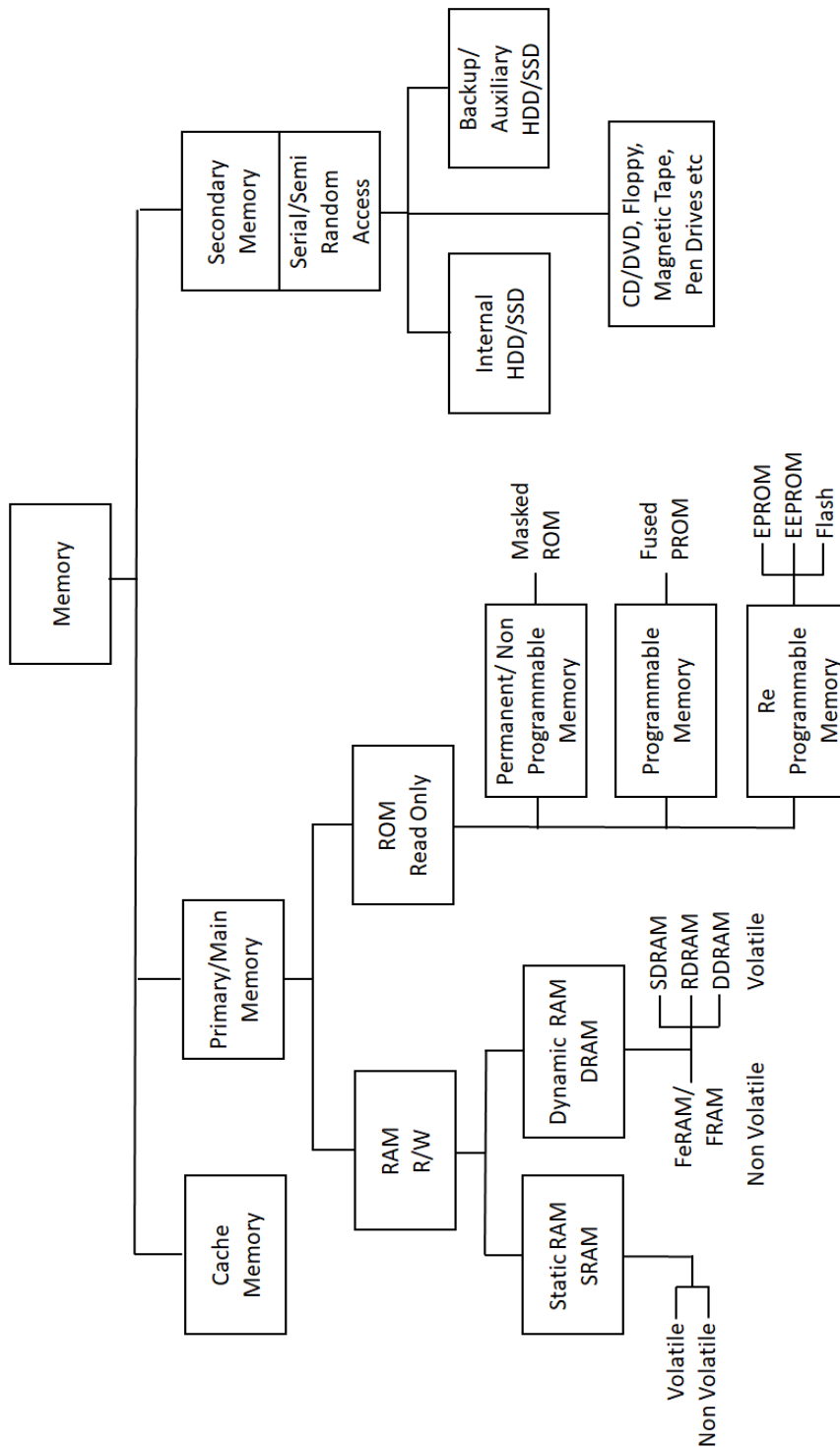


Fig 1.13: Memory Classification

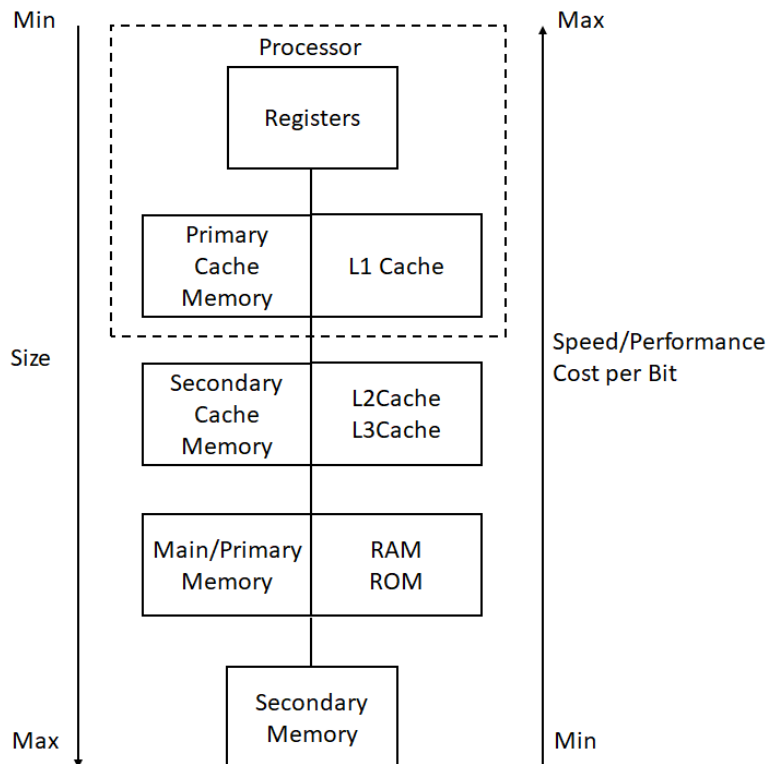


Fig 1.14: Memory Hierarchy

ROM is a Read Only Memory. Commercial applications include ROM that is programmed once. The programming instruction code is later read over thousands of times during the device operation. Development boards incorporate ROM memory that could be erased several times until the bug free release application program is created. EEPROM or Flash memory are used for such applications where the ROM memory can be erased and reprogrammed thousands of times. EEPROM allows access for reading or writing of a single byte. Flash memory allows reading of a single byte; however, it does not allow writing of a single byte, allows only writing of a block of memory. Hence Flash memory has fewer select/access lines and is faster than EEPROM. Flash memories are reprogrammable for over 10,000 times.

RAM is a Random Access Memory, also known as Read and Write Memory. It is normally read or written over thousands of times. This is generally volatile, however there has increasing usage of non-volatile RAMs to store and retain the usage context.

Secondary memory is typically a storage element such as a hard disk like HDD – Hard Disk Drive or an SSD – Solid State Drive which may be internal or external to the computer. Storage media devices such as USB pen drive or CD/DVD ROM drives can be used for external storage and are considered secondary memory.

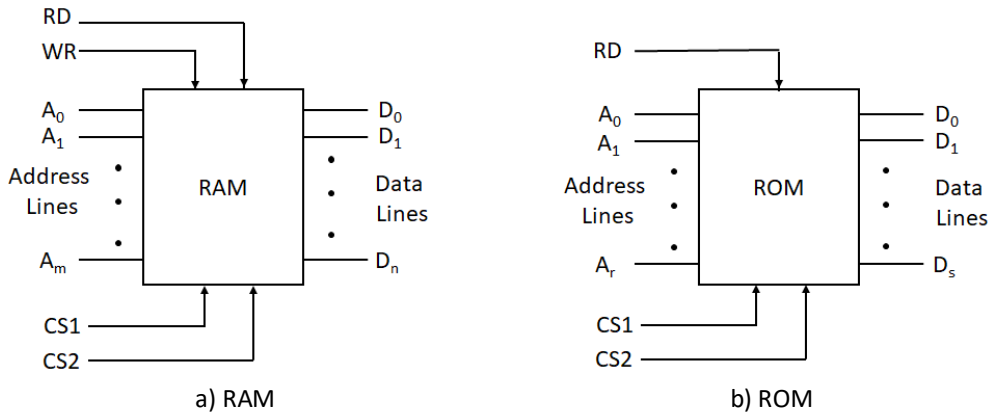


Fig 1.15: Address/Data and Control Signals – Read, Write, Chip Select

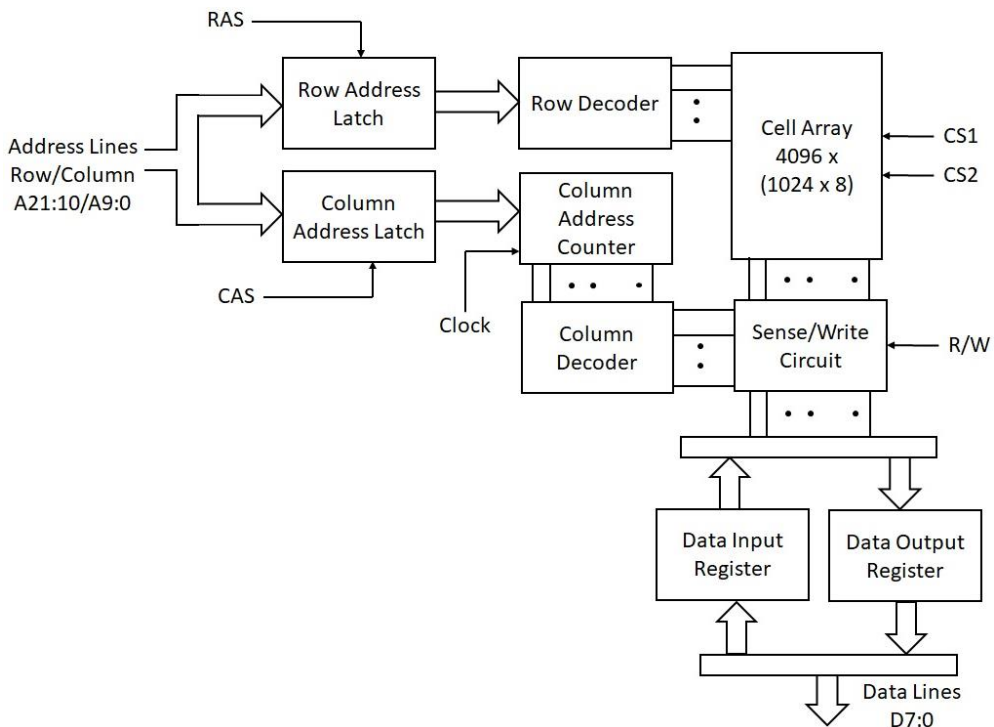


Fig 1.16: Internal Memory Organization

In addition to primary and secondary memory, a small but very fast memory known as cache memory is incorporated to speed up the memory access operations of a processor. This cache memory lies between a processor in the CPU and the main memory RAM/ROM. The cache memory is further arranged to different levels such as L1 cache, L2 cache, L3 cache and so on. The number of levels of cache depends on the design and performance requirements. Since the primary memory consists of a RAM - data memory and ROM – program memory, several designs include different types of cache for data and program

memory such as data cache and instruction cache at L1, L2, L3 and so on. The cache is typically included with in the processor chip. The hierarchy of memory is presented in Fig 1.14.

Memory is interfaced to processor through Address, Data and control/status lines. The Address, Data, Control signal representations for RAM/ROM memory is presented in Fig 1.15. The common control and status signals of memory are RD, WR, PROG, Ready. The RD – Read signal is used to read data from RAM or instruction code from ROM memory. The WR – Write signal is used to write data into RAM memory. A programming interface pin is used to write program instruction code into ROM memory. Memories also contain status pins such as Ready that indicates whether the memory is available for read or write operations. The data value is in the form of a group of binary bits and hence multiple data lines are used to transfer data. They are denoted as D0 to Dn. An application has group of instructions or data that is of several bytes. Several memory locations are required to store the multi byte data or instructions. The memory capacity varies from 128 bytes to several megabytes and Giga bytes. Hence a group of address lines are used to identity a specific memory location and are denoted as A0 to Am etc. A set of memory locations are grouped together to form a block or bank. Several blocks or banks of memory form the actual megabytes or gigabytes of memory. These memory blocks are enabled or selected by using Chip select signals denoted as CS1 or CS2 etc.

An internal organization of memory is presented in Fig 1.16. Each block of memory is internally organized in a matrix form with a set of rows and columns where certain number of address lines are used to select a specific row, and some select a column in that row. To reduce the footprint of the processors IC – Integrated Circuit the address lines and data lines are generally multiplexed. The memory address value is only available on the multiplexed address/data lines for fixed amount of time after which the lines are used for data transfer purposes. Hence memory blocks make use of latches such as row address latch and column address latch to store the address value. This address value is then used by row and column decoders to determine the specific row and column in the block which correspond to a specific memory location. The row and column decoder signals are used to enable the sense/write circuitry to read or write the data from the specific memory location the data input or output register based on the write or read operation.

Virtual Memory:

In computers, the operating systems makes use of a concept called virtual memory. This is implemented through a combination of hardware and software that maps memory addresses used by a program. These addresses are called virtual addresses and are translated to real physical address in computer memory, vice versa through the operating system and address translation hardware in CPU known as MMU – Memory Management Unit. This allows for increased memory addressability, memory isolation, paging,

segmentation and contiguous address space thereby eliminating the need of applications to manage shared memory space between processes.

1.5 Von Neumann & Harvard Architectures

Von-Neumann architecture is a type of architecture where there is a single set of bus structure between CPU and all the other external memory and devices. A common bus structure is used to connect the program memory: ROM, data memory: RAM and peripherals. A typical Von Neumann architecture is shown in Fig 1.17.

A Harvard architecture is a type of architecture where a separate bus structure exists for program memory and data memory i.e. ROM has its own address, data, control bus setup that is different from RAM's address, data, control bus setup. A typical Von Neumann architecture is shown in Fig 1.18.

The advantage of the separate bus structure is that it allows for overlapped instruction fetching and data read or write operations, i.e. the program memory and data memory can be accessed in parallel for acquiring instructions from program memory: ROM and data from data memory: RAM. Since Von-Neumann architecture has only a single set of address, data, control bus setup, overlapped instruction fetch and data operations is not feasible. Hence Harvard architecture is used for high performance such as DSP applications where as Von-Neumann architecture is used where simplicity is desired.

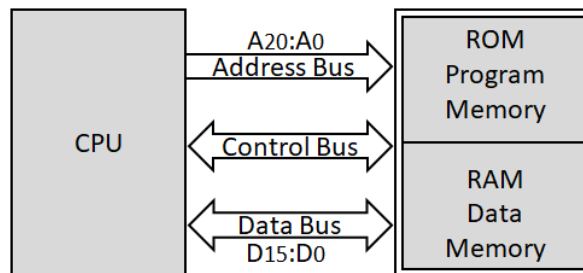


Fig 1.17: Von Neumann Architecture

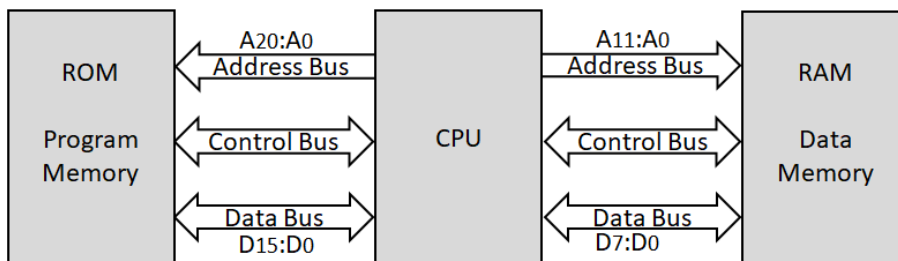


Fig 1.18: Harvard Architecture

1.6 Basic Addressing modes, Instruction formats & Sizes

An application program is a set of instructions. The group of instructions are stored in memory. Each instruction has a specific format which varies based on different factors such as addressing modes, registers, data or address value and the operation.

Addressing Modes:

As seen from the earlier section of bus structure, each device is associated with an address, and data is transferred from one to other. Similarly, from functional units of computer section, we had seen that the CPU consists of registers.

Addressing modes are a form of naming or addressing for information – data or address. As an example, a student can have a registered number, roll number, name and a nick name etc. Similarly, different addressing mechanisms are employed in computer systems. The data can be specified or accessed by CPU in a different number of ways. The data may be accessed through registers, direct memory address, constant immediate data etc. These different ways for accessing data are termed as addressing modes. Some of the commonly used addressing modes are:

- Register Addressing
- Immediate Addressing
- Direct Addressing
- Register Indirect
- Implicit Addressing

Register Addressing

Register addressing mode instructions specify the name of the general purpose or special function registers that holds the required data value. The name of the register forms the operand part of the instruction to perform an operation on data held in the specified register. A typical example for register addressing mode is given below.

MOV A, B → The value of B register is copied to A register in 8085 microprocessor

Immediate Addressing

Immediate addressing mode instructions specify the data value that forms the operand part of the instruction to perform an operation. The data value specified may vary based on the instruction. Some instructions may allow only 8-bit data, some 16-bit or more. In certain devices immediate and direct addressing is differentiated by the operation code itself. i.e. MOV or MVI etc. Some differentiate through special characters. i.e. MOV A, #05H in 8051 or MVI A, 05H in 8085. Typical examples for immediate addressing mode is given below.

MVI A, 05H → 0x05 value is moved to A register in 8085 microprocessor

MOV A, #05H → 0x05 value is moved to A register in 8051 microcontroller

LXI H,9000H → Load HL pair with 16-bit value 9000H in 8085 microprocessor

Direct Addressing

Direct addressing mode instructions specify the address of memory, I/O, Peripheral or memory mapped registers address that hold the data value. The address forms the operand part of the instruction to perform an operation on the respective data in the given address. The address value specified may vary based on the instruction. Some instructions may allow only 8-bit data, some 10-bit or more. Typical examples for direct addressing mode is given below.

MOV R0, 23H → Move a data byte from 8-bit RAM memory address 23H in 8051 microcontroller.

AJMP L1 → L1 forms the lower 11-bit address value of PC in 8051 microcontroller

LDA 9000H → Load A register with a byte value from 16-bit memory address 9000H in 8085 microprocessor.

Register Indirect Addressing

Register Indirect addressing mode instructions specify the name of the register that holds the address of memory, I/O or Peripheral. The data value is present in address specified by the register. This implies that Instruction specifies the register that contains the address of the required data. i.e. Register → Address → Data. The Register forms the operand part of the instruction to perform an operation on the respective data. Some instructions may allow only 8-bit address, some 16-bit or more. Typical examples for register indirect addressing mode is given below.

MOV A, @R0 → Move a data byte from 8-bit internal RAM address specified by R0 register in 8051 microcontroller.

MOVX A, @DPTR → Move a data byte from 16-bit external RAM address specified by DPTR register in 8051 microcontroller.

To understand the concept of Register indirect addressing an analogy of Transport Vehicles Car and Train may be considered. A car can carry people whereas train can carry car as well as people. Here people are considered as data, car as direct address where train may be considered as register indirect addressing. i.e. to get

Implicit Addressing

Implicit addressing mode is a type of addressing mode where the instruction does not contain any operands. A typical example for implicit addressing mode is given below.

STC

→ Set Carry flag in 8086 microprocessor

Instruction formats:

Each instruction is typically associated with a code that is a combination of 8, 16, 24, 32 bits or more. Instruction code typically is a combination of an operation code and an operand. Most common operations are Addition, Subtraction, Multiplication, Jump, Move, Load, Store where the operations are denoted by the operation code ADD, SUB, MUL, JMP, MOV, LOD, STO respectively. The operand typically is a combination of registers, data or address value and addressing mode.

The typical instruction formats are shown in Fig 1.19. The Fig 1.19a presents a generalised format and Fig 1.19b, Fig 1.19c and Fig 1.19d are more specific formats for different addressing modes. In the examples of MOVE, LOAD A etc. in the addressing modes discussed above, MOV, LDA are the opcodes and operands are "A, B" or "A, #05H" or "R0, 23H" etc., where A, B, R0 are registers, #05H is data and 23H is an address.



a) General Instruction Format



b) Register Addressing Mode Format



c) Immediate Addressing Mode Format



d) Direct Addressing Mode Format



e) Indirect Addressing Mode Format



e) Implicit Addressing Mode Format

Fig 1.19: Instruction Formats

Note: Reg1, Reg2, Reg3 represents registers, Data may represent 8, 16 ... unto 32-bit data and address may be 8, 16, unto 32-bit address depending on device and instructions.

Table 1.10: Instruction Coding Data containing Opcode, mode and register select bits.

Operation Code	Operation Select Bits	Addressing Mode	Addressing Mode Select Bits	Register Name	Register Select Bits
ADD	000	Register	00/0	A/R0	00
SUB	001	Immediate	01/0	B/R1	01
MUL	010	Direct	10/0	C/R2	10
JMP	011	Indirect	11/1	D/R3	11
MOV	100				
LOD	101				
STO	110				
MVI	111				

It must be noted that the instruction formats presented here are for general understanding and are not related to any specific device, which actually vary from one device to other. Some of the fields may be optional in some instructions corresponding to the device. Some devices do not include a third register as in PIC, 8085, 8086, 8051. Devices such as ARM uses 3 register syntax. Hence the respective bits would not be part of the instruction format.

Determining the Instruction Hex Code:

Considering the Table 1.10 where operation code select is 3 bits, addressing mode being either 2 or 1 bit and register selection to be 2 bits with a maximum of addressing 2 registers in an instruction. In such a case the MOV A, B instruction may be given by the format Fig 1.19b with two register option and is encoded as follows.

The binary Op Code value for MOV is 100, 0 for register Addressing Mode, and Register select bits 00 for Reg - 2 i.e. A and bits 01 for Reg 1 i.e. B. Hence the code may be equivalent to (LSB) 100 0 00 01 (MSB). Hence the instruction Hex code for MOV A, B is given as 1000 0001 i.e. 81H.

For an ADD A, C instruction the code is as follows. The binary Op Code value for ADD is 000, 0 for register Addressing Mode, and Register select bits 00 for Reg - 2 i.e. A and bits 10 for Reg 1 i.e. B. Hence the code may be equivalent to (LSB) 000 0 00 10 (MSB). Hence the instruction Hex code for MOV A, B is given as (MSB) 0100 0000 (LSB) i.e. 40H.

For an MOV R0, 23H instruction the code is as follows. The binary Op Code value for MOV is 100, 0 for Direct Addressing Mode, and Register select bits 00 for Reg - 1 i.e. R0 and the rest of the two bits are neglected, and the instruction is now a 16-bit or 2 bytes. Hence the first byte forms the hex code and the second by is the actual address 23H. Hence the

code may be equivalent to (LSB) 100 0 00 xx (MSB). Hence the instruction Hex code for MOV R0, 23H is given as xx00 0001 i.e. 01H (for x = 0) followed by address 23H i.e. 0010 0011. The configuring of the ALU for the corresponding code is discussed in the ALU section 1.9.

Instruction sizes:

Instruction size may vary from one instruction to another depending on the operand i.e. size of the data, address or type of the addressing mode, registers etc. Instructions may be of 8-bit (1-byte), 16-bit (2-bytes), 24-bit (3-bytes), 32-bit (4-bytes) size or more. As an example, the 8-bit microprocessor 8085 uses 8-bit or 1byte instructions for register addressing, register indirect addressing and implicit addressing mode instructions. Some immediate addressing mode instructions are 16-bit or 2-byte size, where 1 byte is for operation code and 1 byte is for 8-bit immediate data. Some are 24-bit or 3-byte size, where 1 byte is for operation code and 2 bytes are used for 16-bit immediate data. Similarly, direct addressing mode instructions vary in size based on the size of address.

A general estimation of instruction is given as follows.

MOV A, B	→ 8-bit size as it doesn't contain immediate data or direct address.
MOV A, #05H	→ 16-bit size as it has an 8-bit data of 0x05 value in addition to the operation code and register A selection bits.
LXI H,9000H	→ 24-bit size as it has a 16-bit data of 9000H value in addition to the operation code and register H selection bits.
MOV R0, 23H	→ Move a data byte from 8-bit RAM memory address 23H in 8051 microcontroller.
AJMP L1	→ L1 forms the lower 11-bit address value of PC in 8051 microcontroller
LDA 9000H	→ Load A register with a byte value from 16-bit memory address 9000H in 8085 microprocessor.
MOV A, @R0	→ Move a data byte from 8-bit internal RAM address specified by R0 register in 8051 microcontroller.
MOVX A, @DPTR	→ Move a data byte from 16-bit external RAM address specified by DPTR register in 8051 microcontroller.
STC	→ Set Carry flag in 8085 microprocessor

1.7 Operational Concept of a computer

Having discussed some examples of addressing modes, instructions, instruction formats, sizes we can now discuss the operational concept of a computer. A list some

components of a computer and their functionalities are presented before proceeding to the actual operation of computer. In addition to the basic components already listed as shown in Fig 1.10/Fig 1.11 i.e. Input, Output, Memory, CPU with ALU and Control Unit, we discuss Program Counter – PC, Instruction Register – IR, Decoder, Memory Address Register – MAR and Memory Data Register – MDR. A typical structure showing operational components of a computer is shown in Fig 1.20.

PC – Program Counter

It is a register that holds the address of next instruction that must be executed. This register is incremented for every access of instruction.

MAR – Memory Address Register

It holds the address of memory that is accessed for a read or write operation. It is loaded from PC when an instruction is accessed. It is loaded with an address value of data that must be written or read from a specific location. Any memory location is accessed through this register.

MDR – Memory Data Register

It holds the data that must be written or read from memory. Any data transfer from memory happens through this register.

IR – Instruction Register

It is a register that holds the hex code of the instruction. This data is read from memory and transferred through MDR into IR.

Decoder

The value of IR register is read and decoded to determine the kind of operation that must be performed by the control unit.

Control Unit

It is responsible for enabling, disabling, generating the corresponding signals to activate the required components at the specific cycle times. It configures the ALU based on the instruction register value.

Execution Unit - ALU

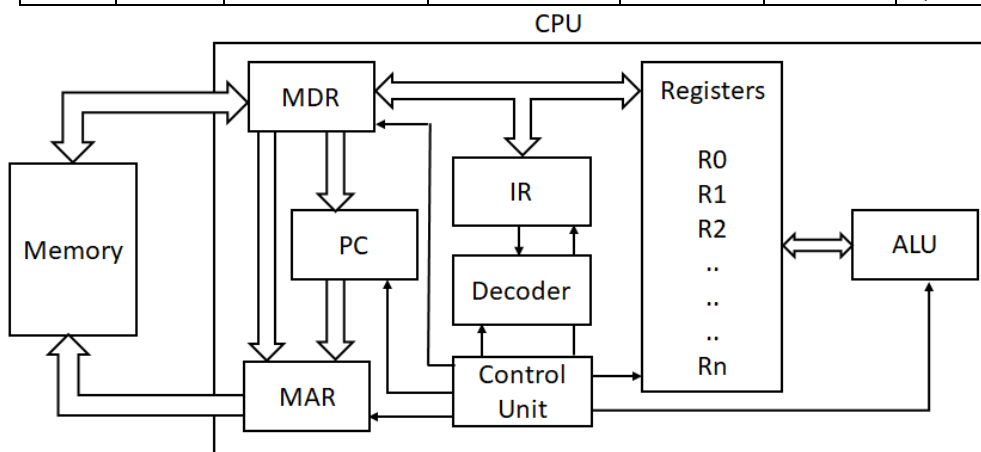
Based on the actual instruction the ALU is configured by the control unit. It performs arithmetic operations like addition, subtraction increment etc. or logical operations like AND, OR etc.

We consider the following program to demonstrate the operational concept of computer. It performs addition of two 8-bit immediate data repetitively in an infinite loop.

```
L1    MVI    A, 05H  → Move Value of 05 to A. i.e. A = 05H
      MVI    B, 03H  → Move Value of 03 to B. i.e. B = 03H
      ADD    A, B    → Add register A and B and store the result in destination
                        i.e. A. so A = 05+03 = 08
      STO    A, 10H  → Store the value of A into memory at address 10H
```

Table 1.11: Repetitive Addition Program instruction coding, size and memory locations.

OP Code	Oper- and	Binary Format LSB to MSB	Binary Format MSB to LSB	Data/ Address	Hex Code	Mem Address
MVI	A, 05H	LSB 111 0 00xx	xx00 0111	0000 0101	07H 05H	0,1
MVI	B, 03H	LSB 111 0 01xx	xx10 0111	0000 0011	27H 03H	2,3
ADD	A, B	LSB 000 0 0001	1000 0000		80H	4
STO	A, 10H	LSB 110 0 00xx	xx00 0011	0001 0000	03H 10H	5,6
JMP	L1	LSB 011 0 xxxx	xx00 0110	0000 0000	06H 00H	7,8

**Fig 1.20: Operational components of a computer**

The operation of the computer for the above mentioned program can be described through the following steps. A detailed structure showing interconnection of operational components in a computer is shown in Fig 1.21.

The execution of an application program is started by loading the PC with the starting address of the program. To execute this instruction, the PC must be loaded with a value of 0000H so that execution starts from 0H. When the PC is loaded with starting address, the address from it is copied to the memory address register – MAR. This is achieved by an internal control read signal to the PC which places the PC data on the bus. It is followed by an internal control write signal to the MAR register, which allows write to MAR register and allows for activating the respective location in the memory.

The data in the given memory location (i.e. Hex Code) is copied on to the data bus and to memory data register – MDR when the control unit CU generates an external read - RD signal to the memory. It is followed by an external write – WR signal to the MDR register. The PC is incremented by 1 for every access of 1 instruction byte. The Control Unit then activates internal signal such that the data from MDR is copied to the instruction register -

IR. This is achieved by an internal read signal to the MDR which places the MDR data on the bus. It is followed by an internal write signal to the IR register.

When the decoder is activated, the value in IR register is then decoded. The decoders information helps the control unit in determining the next sequence of steps that must be made for specific instruction.

The summary of steps common to all the instructions is given as follows.

- Load the PC with the starting address of the application program. (Only at program start)
- Copy the contents of PC to MAR using internal control signals
- Copy the contents of respective memory location (Hex Code) to MDR using the external control signals RD and internal write signal.
- Copy the contents of MDR to IR using internal control signals.
- Increment PC by 1 for every 1-byte access of memory.
- Decode the value in IR register to help the CU determine the next sequence of steps specific to the respective instructions.

Instruction 1: MVI A, 05H

This instruction copies the value of 05H to register A and is the first instruction in the program. It is stored as Hex Code with values of 07H and 05H in locations 0 and 1 of memory. The common sequence of steps for all the instructions has been presented earlier in this section. The specific sequence of steps that vary based on instruction are presented here for instruction 1.

After decoding of the Hex code 07H in IR by the decoder (that was brought from memory to IR) the CU is made aware that the following byte in memory 05H is a data byte. Hence a memory read operation cycle is initiated. i.e.

- Copy the contents of PC to MAR using internal control signals
- Copy the contents of respective memory location to MDR using the external control signal RD to memory and internal write signal to MDR.
- Increment PC by 1 for every 1-byte access of memory.
- Copy the contents of MDR to A register using internal control signals i.e. read signal to MDR and write signal to A register.

Instruction 2: MVI B, 03H

This instruction copies the value of 03H to register B and is the second instruction in the program. It is stored as Hex Code with values of 27H and 03H in locations 2 and 3 of memory. The set of instructions for instruction 2 are same as instruction 1 with the exception that B register is used instead of A register, hence the hex code is 27H instead of 07H and the data value is 03H instead of 05H. The rest of the steps remain the same.

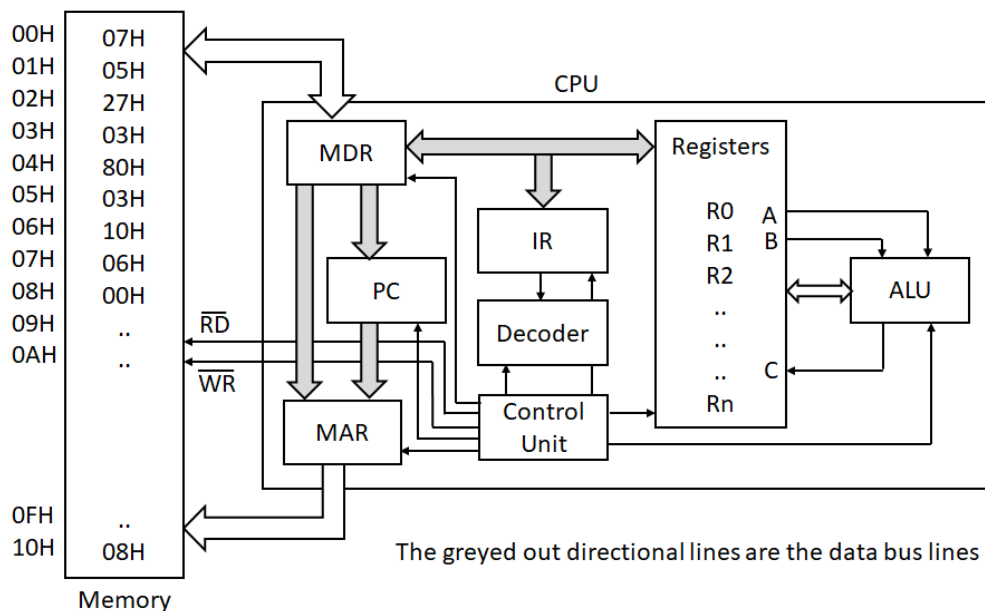


Fig 1.21: Operational concept of a computer for repetitive addition program

Instruction 3: ADD A, B

This instruction reads the A and B registers and gets the value of 05H and 03H to perform the addition on A and B and stores the result in A register. The figure shows ALU result sent to C to indicate general operation which is present in 3-register structure such as ARM. For demonstration operational concept we assume the ALU result to be stored back to A register. The ADD A, B instruction is stored as Hex Code with value of 80H in location 4H of memory. **The common sequence of steps for all the instructions have been presented earlier in this section.** The specific sequence of steps that vary based on instruction are presented here for instruction 3.

After decode of the Hex code 80H in IR by the decoder (that was brought from memory to IR) the CU is made aware that addition operation must be performed by reading A and B register and store the result back to A register.

Hence a registers read, addition operation & a register write operation must take place, i.e.

- Read the register A and register B using internal read control signals and make it available for ALU
- Enable the ALU using internal control signal and configure to perform addition operation. The configuring of ALU is presented later in ALU configuration section.
- After addition is performed in the ALU, the result 08H is stored in register A (or C) by an internal control write signal to A (or C) register.

Instruction 4: STO A, 10H

This instruction stores the A registers content on to memory location 10H. The STO A, 10H instruction is stored as Hex Code with value of 03H and 10H in location 5H and 6H of memory. The common sequence of steps for all the instructions have been presented earlier in this section. The specific sequence of steps that vary based on instruction are presented here for instruction 4.

After decode of the Hex code 03H in IR by the decoder (that was brought from memory to IR) the CU is made aware that the following byte in memory 6H is the address location 10H where data must be stored. Hence a memory read operation cycle is initiated for getting the stored address location 10H from 6H. It is followed by a register read, and a memory write operation, i.e.

Memory Read Operation:

- Copy the contents of PC (6H) to MAR using internal control signals
- Copy the contents (10H) of memory location 6H (Hex Code) to MDR using the external control signal RD to memory and internal write signal to MDR.
- Increment PC by 1 for every 1-byte access of memory.
- Copy the contents of MDR (10H) to MAR register using internal control signals i.e. read signal to MDR and write signal to MAR register.

Memory Write Operation:

- Further the A (or C) register is enabled through an internal read signal which places the data on data bus and MDR register is written by enabling through an internal write signal to store the A register value 08H in MDR register.
- The addition result in MDR (08H) is written to 0x10H (pointed by MAR) memory location when the CU generates an external write signal WR to memory and an internal read signal to MDR register.

Instruction 5: JMP L1

This instruction loops back to the first instruction stored at memory location 0H to repeat the set of instructions indefinitely. The JMP L1 instruction is stored as Hex Code with value of 06H and 00H in location 7H and 8H of memory. The common sequence of steps for all the instructions have been presented earlier in this section. The specific sequence of steps that vary based on instruction are presented here for instruction 5.

After decode of the Hex code 06H in IR by the decoder (that was brought from memory to IR) the CU is made aware that the following byte in memory 8H is the address location where the jump has to be made. Hence a memory read operation cycle is initiated for getting the stored jump address location 00H from 8H memory location, i.e.

Memory Read Operation:

- Copy the contents of PC (8H) to MAR using internal control signals

- Copy the contents of respective memory location 8H to MDR using the external control signal RD to memory and internal write signal to MDR.
- Increment PC by 1 for every 1-byte access of memory.
- Copy the contents of MDR (00H) to PC register using internal control signals i.e. read signal to MDR and write signal to PC register so that next instruction is fetched from 00H.

This simple program has been used to demonstrate the operational concept through the different operation sequences that must be performed by the CU for execution of the application program.

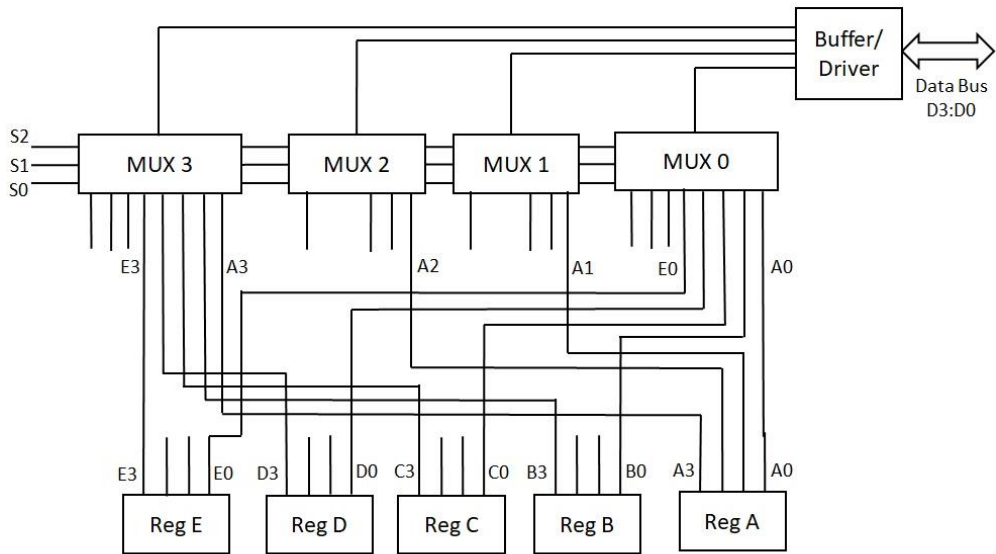
1.8 Register Bus Structure

From the earlier section it can be observed that the processor contains multiple registers. Hence a bus structure is required for connecting all the registers for transferring data from one of the selected registers to the ALU, bus or other components. This section demonstrates the design of a general register bus structure and its application for ALU.

Let's consider the case of setup consisting of 5 registers, with each register consisting of 4 bits. Since the data present in the register is of 4-bits, the required size of the bus is also 4-bits. Similarly, the structure is supposed to incorporate 5 registers and one of the 5 registers must be selected. Hence a selector is required to select among the 5 registers. The most commonly used selector is the multiplexer. Hence a multiplexer of 8:1 may be used with 5 inputs from 5 registers and leaving the other 3 disconnected. Since the bus required is only 4 bits, we require a total of 4 multiplexers, one for each bit with each capable of selection among 8:1. A connection arrangement of the same is presented here in Fig 1.22 that acts as a bus structure for 5 registers with each register consisting of 4 bits.

Here the bit 0's of all the registers is connected to MUX0 so that it provides the bit 0 of the selected register to the bus. Similarly bit 1's of all the registers is connected to MUX1 and so on with bit 3's of all registers to MUX3. All the multiplexers share common select lines to select the corresponding registers bits to output on the data bus.

The Fig 1.22 presents a single bus structure. However, a computer requires more than 1 bus structure. An arrangement requiring 3 bus structures for connecting registers to ALU is shown in Fig 1.23. This design incorporates 8 registers. The bus structure 1 provides the source 1 data for the ALU, while the bus structure 2 provides the source 2. The bus structure 3 provides for storing the result in destination register. Some systems use only 2 bus structures like the one presented in operational concept. Ex: $A = A + B$. In such a case a bidirectional bus structure may be used since destination is same as source 1. For operations such as $C = A + B$, three bus structures are used.



Note: F,G,H inputs of MUX are left unused as only 5 registers are present

Fig 1.22: Multiplexer based bus Structure for five 4-bit registers

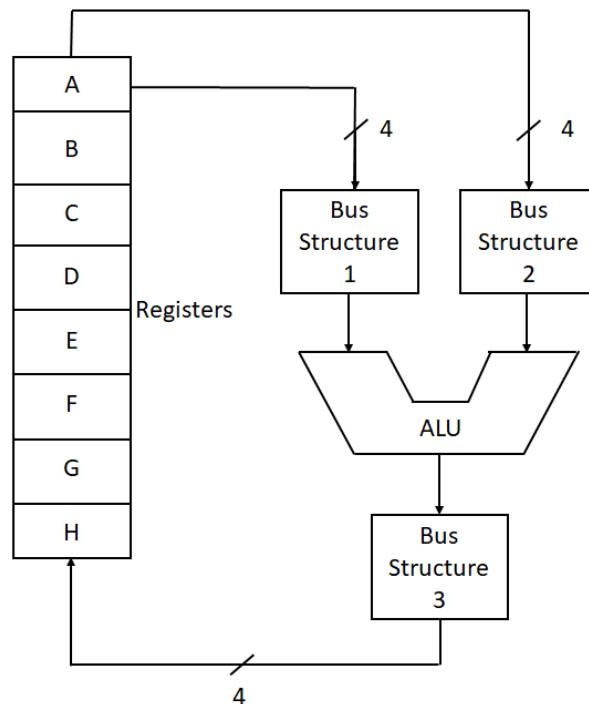


Fig 1.23: Bus Structure application to ALU

The register select and corresponding binary, HEX code bits for the three bus structure circuit is presented in Table 1.12. For a source 1 and source 2 as E and F registers,

and destination as H register the binary select HEX code is xxx 1 11 10 0 101 i.e. 1E5H. The binary select bits for source 1 is QQQ i.e. RSA210 corresponding to register E is 100. The binary select bits for source 2 is RRR i.e. RSB210 corresponding to register F is 101. The binary select bits for destination is PPP i.e. RD210 corresponding to register H is 111. Hence a total of 9 bits are used for 3 register selections. Additionally, 3 bits are kept reserved as denoted by xxx in table 1.12.

Table 1.12: Three Bus Structure Register Select & corresponding Binary, HEX code bits.

Register	Register Select Bits	Hex Code	OP Code Binary SZxxx-(RD210) (RSA210) (RSB210)
A	000	0	Z xxx PPP QQQ RRR
B	001	1	
C	010	2	
D	011	3	
E	100	4	
F	101	5	
G	110	6	
H	111	7	

Note: Here RD210 corresponds to PPP, it represents bits for destination, RSA210 and RSB210 correspond to QQQ & RRR, they represent bits for source 1 and 2. The Sources 1, 2 and destination could be any of the registers A to H.

1.9 Configuration and design of ALU.

The CPU typically consists of an ALU, Shift/Rotate unit and also contains multiple register bus structures. From the section of operational concepts of a computer we have seen that the ALU is configured by the control unit to perform different arithmetic or logical operations. Similarly, other units are also configured to perform the respective operations.

1.9.1 Configuration and design of ALSU.

This section deals with design and configuration of ALU. The most common arithmetic operations are Addition, Subtraction, Increment, Decrement etc. Similarly, common logical operations are AND, OR, XOR, XNOR etc. Similarly shift related operations such as arithmetic shift, logical shift and rotate operations are common in computing devices.

The basic combinational circuits such as adders and subtractors are already presented in first section. Here we build up on those basic components in the design of ALU. The design of ALU presented here is centred around a basic Ripple Carry Adder. A typical 4-bit ripple carry adder representation using full adders is shown in Fig 1.24 where A3:0 and B3:0 form the two 4-bit numbers A and B.

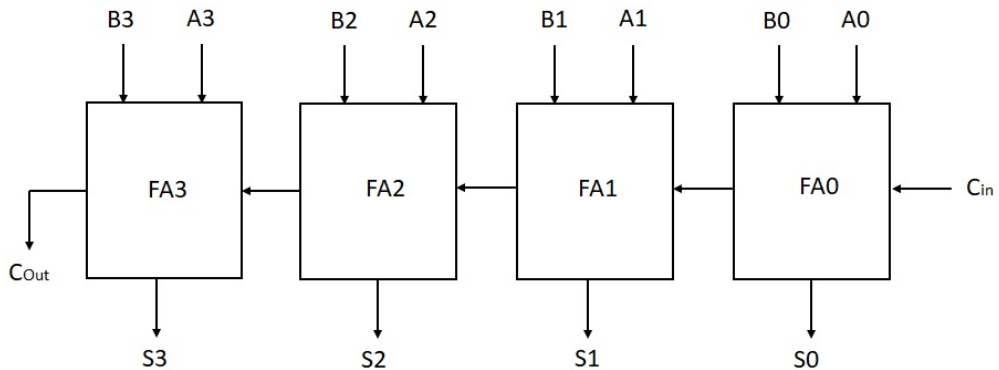


Fig 1.24: Ripple Carry Adder

The ripple carry adder shown in Fig 1.24 is only suitable for performing addition operations. However, for performing subtraction and to provide for other functionalities, the circuit shown in Fig 1.24 is modified. A 3-bit configuration of the modified circuit is shown in Fig 1.25.

Arithmetic Operations.

The circuit of Fig 1.25 incorporates most of the required functionalities such as addition, Subtraction of B, 1's and 2's complement of B, Increment, Decrement, MOV etc.

Addition:

The addition operation is achieved by connecting the switch s12 to position 0, switch S34, S56 & S9 to position 0 and switch S78 position to 3. Result = $A + B$

Addition with Carry:

The addition with carry operation is achieved by connecting the switch S34, S56 & S9 to position 0 and switch S78 position to 0. Result = $A + B + C_{in}$

Subtraction:

The subtraction operation is achieved connecting the switch S34 to position 1, switch S56 & S9 to position 0 and switch S78 position to 2. Result = $A + \underline{B} + 1$. Legacy devices does not allow subtraction of A i.e. $B - A$ is not possible. Similarly, B cannot be moved to the output because switch arrangement S12 with V_{CC} , GND, NOT connections are not provided for input A. Hence a generalised circuit shown in Fig 1.26 is used which is flexible to provide different variations such as $A - B$ or $B - A$ etc.

Subtraction with Borrow:

The subtraction with borrow operation is achieved connecting the switch S34 to position 1, switch S56 & S9 to position 0 and switch S78 position to 0. Hence the result is given as $A + \underline{B} + 1 - C_{in}$ which results in $A + \underline{B}$ when C_{in} is 1 and a normal subtraction result $A + \underline{B} + 1$ when there is no borrow. Some devices assume that C_{in} is 0 when there is a borrow

and 1 when there is no borrow. However, intel devices like 8085, 8085 and 8051 represents Cin as 1 when there is a borrow and 0 when there is no borrow.

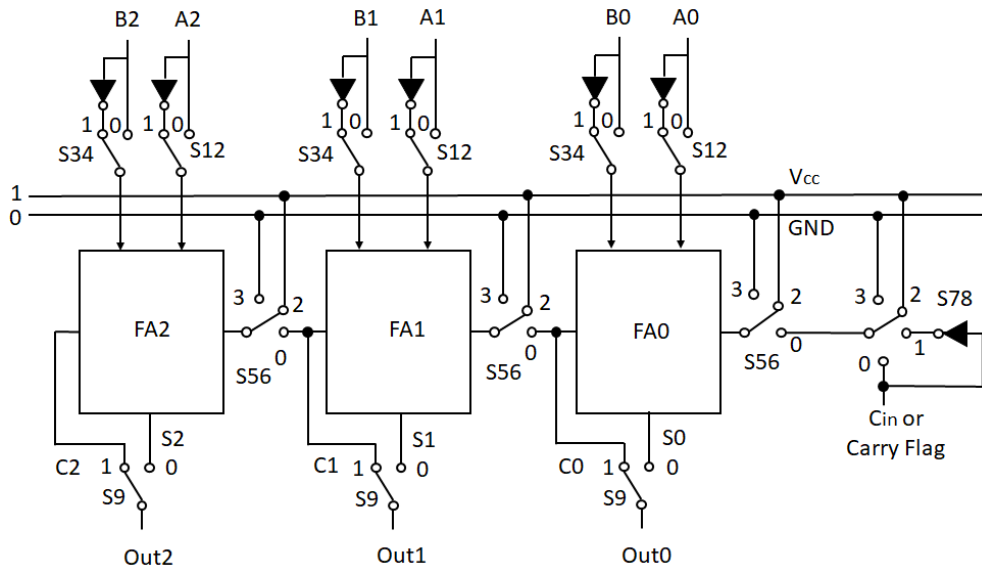


Fig 1.25: Arithmetic & Logical Circuit – Switch based

Move - MOV:

The move operation of A to the output is achieved by connecting the switch S34, S56 to position 3, switch S9 to position 0 and S78 position to 3. Ex: MOV Destination, A.

Increment:

The increment operation of A is achieved by connecting the switch 12 to position 0, switch S34 to position 3, switch S56 & S9 to position 0 and S78 position to 2.

Decrement:

The decrement operation i.e. Decrement by 1 is achieved by adding all 1's to the given number in A. 1's complement of 1 is 1110 and 2's complement of 1 is 1's complement + 1. Hence $1110 + 1 = 1111$.

Decimal No	1	0101 (5)	0101 (5)
Binary Form	0001	- 1	<u>+1111 (-1)</u>
1's complement	1110	0100 (4)	0100 (4)
	<u>+1</u>		
	1111		

The decrement is achieved by connecting the switch S12 to position 0, switch S34 to position 2, switch S56 to position 0, S9 switch to position 0 and switch S78 to position 3.

Logical Operations.

The circuit of Fig 1.26 incorporates most of the required logical operation functionalities and may be used to perform the logical operations such as AND, OR, XOR, XNOR, NOT, BUFFER etc. To demonstrate the logical operations using ripple carry adder circuit, the corresponding adder expressions are considered.

The Half Adder Sum and Carry expressions are

$$H_k = A_k \oplus B_k = A_k \bar{B}_k + \bar{A}_k B_k \quad (1.1)$$

$$C_k = A_k \cdot B_k \quad (1.2)$$

Here k represents the number of the stage.

The Full Adder Sum and Carry expressions are

$$\text{Sum} = H_k \oplus C_{k-1} = C_{k-1} \bar{H}_k + \bar{C}_{k-1} H_k \quad (1.3)$$

$$\text{Carry} = A_k \cdot B_k + H_k \cdot C_{k-1} \quad (1.4)$$

XOR: Exclusive – OR

The XOR operation is demonstrated considering eq. 1.3. When the input carry i.e. C_{k-1} is set to 0 the eq. 1.3 reduces to H_k which is XOR operation of A_k, B_k . The XOR operation of A, B is achieved by connecting the switch S12, S34 to position 0, switch S56 to position 3, switch S9 to position 0 and S78 position to 3. Ex: XOR A, B.

XNOR: Exclusive – NOR

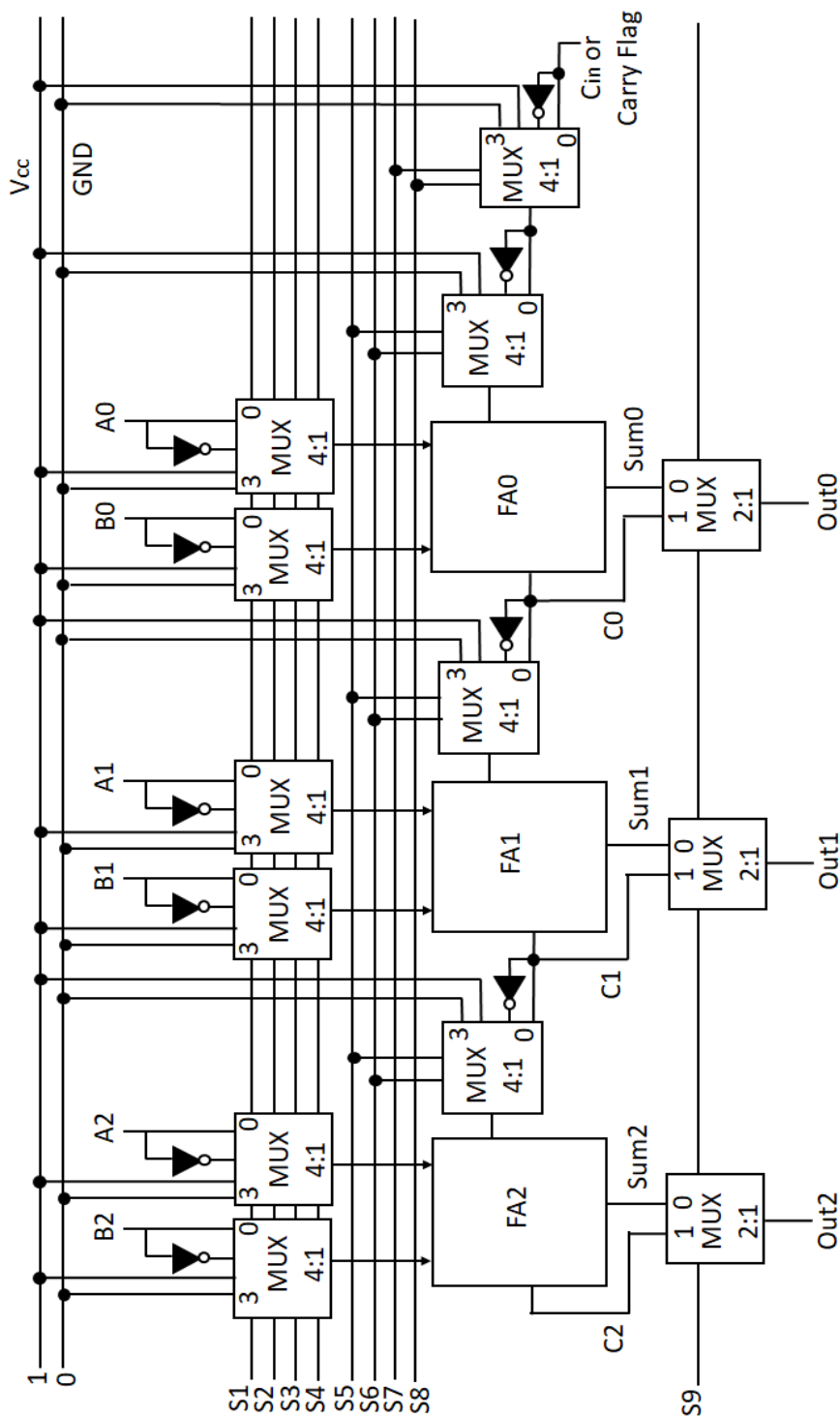
The XNOR operation is demonstrated considering eq. 1.3. When the input carry i.e. C_{k-1} is set to 1 the eq. 1.3 reduces to $\bar{H}_k$ which is XNOR operation of A_k, B_k . The XNOR operation of A, B is achieved by connecting the switch S12, S34 to position 0, switch S56 to position 2, switch S9 to position 0 and S78 position to 2. Ex: XNOR A, B.

AND:

The AND operation is demonstrated considering eq. 1.4. When the input carry i.e. C_{k-1} is set to 0, the eq. 1.4 reduces to $A_k \cdot B_k$ which is AND operation of A_k, B_k . The AND operation of A, B is achieved by connecting the switch S12, S34 to position 0, switch S56 to position 3, switch S9 to position 1 and S78 position to 3. Ex: AND A, B.

OR:

The OR operation is demonstrated considering eq. 1.4. When input carry i.e. C_{k-1} is set to 1, the eq. 1.4 reduces to $A_k \cdot B_k + H_k$ which is OR operation of A_k, B_k . The OR operation of A, B is achieved by connecting the switch S12, S34 to position 0, switch S56 to position 3, switch S9 to position 1 and S78 position to 2. Ex: OR A, B.



Note: S1, S2 select lines are for input A and S3, S4 are for input B.

For shift and rotate operations, additional circuitry with shift registers and multiplexers must be included.

Fig 1.26: Arithmetic & Logical Circuit – Generalized: Multiplexer based

Table 1.13: Arithmetic/Logical operations & corresponding Binary, HEX code bits.

OP Code	Expression, D: Destination	Description	OP Code Binary SY-xxx9 87 65 43 21	Hex Code
MOV	$D = A$	Move/Pass A	Y xxx0 11 11 11 00	00FC
ADD	$D = A + B$	Add A, B	Y xxx0 11 00 00 00	00C0
ADC	$D = A + B + Cy$	Add A, B with Carry	Y xxx0 00 00 00 00	0000
SUB	$D = A - B$	Subtract i.e. $A - B$, $D = A + \underline{B} + 1$	Y xxx0 10 00 01 00	0084
SBB	$D = A - B - Cy$	Subtract i.e. $A - B - Cy$ $D = A + \underline{B} + 1 - Cy$ If $Cy = 0$ does SUB, If $Cy = 1$ does SUBB	Y xxx0 01 00 01 00	0044
INC	$D = A + 1$	Move/Pass $A + 1$	Y xxx0 10 00 11 00	008C
DEC	$D = A - 1$	Move/Pass $A - 1$ $D = A + \text{all } 1\text{'s}$	Y xxx0 11 00 10 00	00C8
NOT	$D = \overline{A}$	Move/Pass $\overline{A}$	Y xxx0 11 11 11 01	00FD
NEG	$D = \overline{A} + 1$	Move/Pass $\overline{A} + 1$	Y xxx0 10 00 11 01	008D
XOR	$D = A \oplus B$	XOR A, B	Y xxx0 11 11 00 00	00F0
XNOR	$D = A \oplus B$	XNOR A, B	Y xxx0 11 10 00 00	00E0
AND	$D = A . B$	AND A, B	Y xxx1 11 11 00 00	01F0
OR	$D = A + B$	OR A, B	Y xxx1 11 10 00 00	01E0

1's Complement - NOT:

The NOT operation of A is achieved by connecting the switch S12 to position 1, switch S34, S56 to position 3, switch S9 to position 0 and S78 position to 3. Ex: NOT A.

2's Complement - NEG:

The NEG operation of A is achieved by connecting the switch S12 to position 1, switch S34 to position 3, switch S56, S9 to position 0 and S78 position to 2. Ex: NEG A.

The summary of different operations and corresponding opcode binary & hexadecimal code bits for the given circuit of Fig 1.26 is presented in Table 1.13.

1.9.2 Configuration and design of Shift/Rotate Unit.

In addition to the ALU unit, a shift unit is generally incorporated into the processor. Devices like ARM processors employs a shifter unit at one of the ALU inputs. i.e. the input B applied to ALU may be a shifted, rotated or an un-shifted input.

The circuit of Fig 1.27 demonstrates the structure of an ALU with a shift/rotate unit. The common shift/rotate operations are either a right shift or left shift with arithmetic, logical shifts, or left or right rotate with or without using a carry. A detailed schematic of a 3-bit shift/rotate unit is presented in Fig 1.28. The summary of different operations and corresponding opcode binary & hexadecimal code bits for the given circuit of Fig 1.28 is presented in Table 1.14.

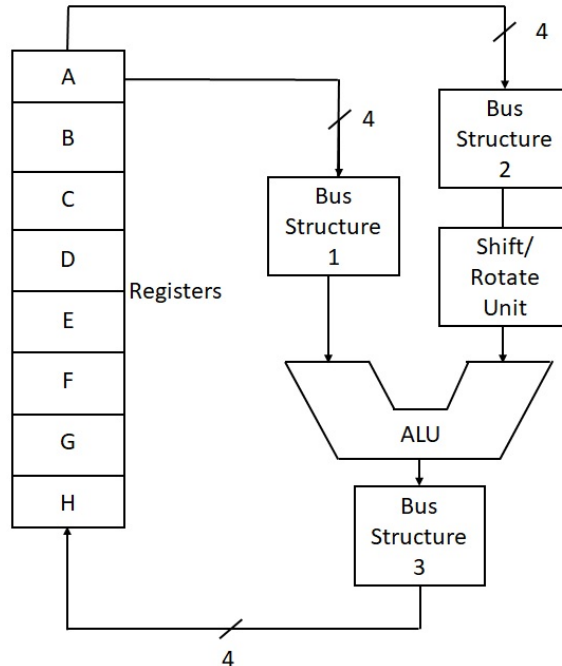


Fig 1.27: Arithmetic Logic & Shift Unit

ASR – Arithmetic shift right:

In arithmetic right shift, the LSB bit is discarded, and all the higher bits are shifted to the right. The old MSB bit is copied and replicated into the MSB of new shifted data bits. For an 8-bit data, represented by bit 0 to bit 7, the bit 7:1 is copied into bit 6:0. The original bit 0 is discarded and earlier bit 7 is copied into the new bit 7. Examples of the same is presented in Table 1.14.

ASRC – Arithmetic shift right with Carry:

The arithmetic right shift with carry is similar to arithmetic right shift with the exception that LSB bit 0 is not discarded, instead it is moved to carry flag. Examples of the same is presented in Table 1.14.

ASL – Arithmetic shift Left:

In arithmetic left shift, the MSB bit is discarded, and all the lower bits are shifted to the left. A '0' is copied into the LSB bit. For an 8-bit data, represented by bit 0 to bit 7, MSB bit 7 is discarded and the bit 6:0 is copied into bit 7:1 and the bit 0 is copied with value of 0. Examples of the same is presented in Table 1.14.

ASLC – Arithmetic shift Left with Carry:

The arithmetic left shift with carry is similar to arithmetic left shift with the exception that MSB bit 7 is not discarded, instead it is moved to carry flag. Examples of the same is presented in Table 1.14.

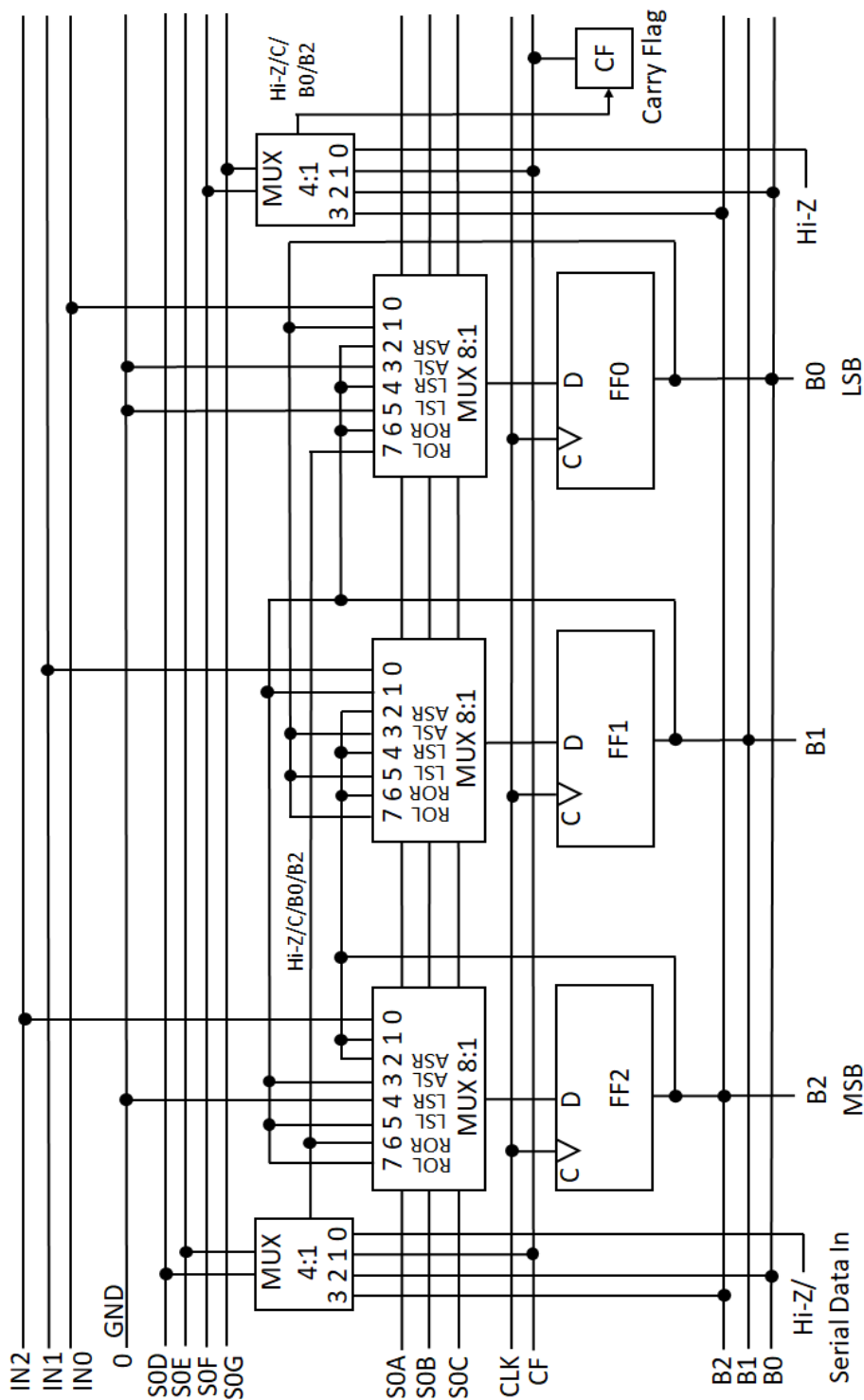


Fig 1.28: Shift/Rotate Unit

LSR – Logical shift right:

In logical right shift, the LSB bit is discarded, and all the higher bits are shifted to the right. The old MSB bit is cleared to 0. For an 8-bit data, represented by bit 0 to bit 7, the bit 7:1 is copied into bit 6:0. The original bit 0 is discarded and earlier bit 7 is cleared to 0. Examples of the same is presented in Table 1.14.

LSRC – Logical shift right with Carry:

The logical right shift with carry is similar to logical right shift with the exception that LSB bit 0 is not discarded, instead it is moved to carry flag. Examples of the same is presented in Table 1.14.

Table 1.14: Shift/Rotate operations & corresponding Binary, HEX code bits.

OP Code	Description	OP Code Binary S0-xABC DEFG	Hex Code	Before	After
No Shift	No Shift, Pass Data as it is	0000 0000	00	0111 1011	0111 1011
ASR	Arithmetic Shift Right	1010 0000	A0	a) 1111 0011 b) 0111 1011	a) 1111 1001 b) 0011 1101
ASRC	Arithmetic Shift Right with Carry	1010 0010	A2	0111 1011 CY = 0 or 1	0011 1101 CY = 1 LSB
ASL	Arithmetic Shift Left	1011 0000	B0	a) 1111 0011 b) 0111 1011	a) 1110 0110 b) 1111 0110
ASLC	Arithmetic Shift Left with Carry	1011 0011	B3	0111 1011 CY = 0 or 1	1111 0110 CY = 0 MSB
LSR	Logical Shift Right	1100 0000	C0	a) 1111 0011 b) 0111 1011	a) 0111 1001 b) 0011 1101
LSRC	Logical Shift Right with Carry	1100 0010	C2	0111 1011 CY = 0 or 1	0011 1101 CY = 1 LSB
LSL	Logical Shift Left	1101 0000	D0	a) 1111 0011 b) 0111 1011	a) 1110 0110 b) 1111 0110
LSLC	Logical Shift Left with Carry	1101 0011	D3	0111 1011 CY = 0 or 1	1111 0110 CY = 0 MSB
ROR	Rotate Right	1110 1000	E8	a) 1111 0010 b) 0111 1011	a) 0111 1001 b) 1011 1101
RCR	Rotate Right with Carry	1110 0110	E6	a) 0111 1011 CY = 0 b) 0111 1010 CY = 1	0011 1101 CY = 1 LSB 1011 1101 CY = 0 LSB
ROL	Rotate Left	1111 1100	FC	a) 1111 0010 b) 0111 1011	a) 1110 0101 b) 1111 0110
RCL	Rotate Left with Carry	1111 0111	F7	a) 0111 1010 CY = 1 b) 1011 1011 CY = 0	a) 1111 0101 CY = 0 MSB b) 0111 0110 CY = 1 MSB

LSL – Logical shift Left:

In logical left shift, the MSB bit is discarded, and all the lower bits are shifted to the left. A '0' is copied into the LSB bit. For an 8-bit data, represented by bit 0 to bit 7, MSB bit 7 is discarded and the bit 6:0 is copied into bit 7:1 and the bit 0 is copied with value of 0. Examples of the same is presented in Table 1.14.

LSLC – Logical shift Left with Carry:

The logical left shift with carry is similar to logical left shift with the exception that MSB bit 7 is not discarded, instead it is moved to carry flag. Examples of the same is presented in Table 1.14.

ROR – Rotate right:

In rotate right operation, the LSB bit is copied, all the higher bits are shifted to the right and the earlier saved LSB bit is moved to MSB. For an 8-bit data, represented by bit 0 to bit 7, the bit 0 is copied, bit 7:1 is copied into bit 6:0. The original saved LSB bit 0 is moved to MSB bit 7. Examples of the same is presented in Table 1.14.

RCR – Rotate right with Carry:

In rotate right with carry operation, the LSB bit is copied, all the higher bits are shifted to the right, carry bit is copied to MSB and the earlier saved LSB bit is moved to Carry. For an 8-bit data, represented by bit 0 to bit 7, the bit 0 is copied, bit 7:1 is copied into bit 6:0 and Carry bit is moved to bit 7. The original saved bit 0 is moved to carry. Examples of the same is presented in Table 1.14.

ROL – Rotate Left:

In rotate left operation, the MSB bit is copied, all the lower bits are shifted to the left and the earlier saved MSB bit is moved to LSB. For an 8-bit data, represented by bit 0 to bit 7, the bit 7 is copied, bit 6:0 is copied into bit 7:1. The original saved MSB bit 7 is moved to LSB bit 0. Examples of the same is presented in Table 1.14.

RCL – Rotate Left with Carry:

In rotate left with carry operation, the MSB bit is copied, all the lower bits are shifted to the left, the carry bit is moved to LSB and the earlier saved MSB bit is moved to carry. For an 8-bit data, represented by bit 0 to bit 7, the bit 7 is copied, bit 6:0 is copied into bit 7:1 and carry bit is moved to bit 0. The original saved MSB bit 7 is moved to carry. Examples of the same is presented in Table 1.14.

1.9.3 Configuration and design of combined ALU, Shift/Rotate and multi bus structure unit.

The different ALU operations may be combined with shift rotate operations. Additionally, it requires the circuit design of the form shown in Fig 1.27 for the operation of the different units. The Table 1.15 presents the summary of various operations/instructions

and corresponding binary, HEX code bits separately. The Table 1.16 presents the combined hex code considering the three-bus structure presented in Fig 1.27 which uses ALU of Fig 1.26 and shift rotate unit of Fig 1.28. The Table 1.17 presents the hex code additionally considering register select bits of the three-bus structure for addition operation with and without shift operation.

Table 1.15: Summary of various instructions & corresponding Binary, HEX code bits.

S. No	OP Code	OP Code Binary SY-xxx9 87 65 43 21	Hex Code	S. No	OP Code	OP Code Binary S0-xABC DEFG	Hex Code
1	MOV	Y xxx0 11 11 11 00	00FC	21	No Shift	0000 0000	00
2	ADD	Y xxx0 11 00 00 00	00C0	22	ASR	1010 0000	A0
3	ADC	Y xxx0 00 00 00 00	0000	23	ASRC	1010 0010	A2
4	SUB	Y xxx0 10 00 01 00	0084	24	ASL	1011 0000	B0
5	SBB	Y xxx0 01 00 01 00	0044	25	ASLC	1011 0011	B3
6	INC	Y xxx0 10 00 11 00	008C	26	LSR	1100 0000	C0
7	DEC	Y xxx0 11 00 10 00	00C8	27	LSRC	1100 0010	C2
8	NOT	Y xxx0 11 11 11 01	00FD	28	LSL	1101 0000	D0
9	NEG	Y xxx0 10 00 11 01	008D	29	LSLC	1101 0011	D3
10	XOR	Y xxx0 11 11 00 00	00F0	30	ROR	1110 1000	E8
11	XNOR	Y xxx0 11 10 00 00	00E0	31	RCR	1110 0110	E6
12	AND	Y xxx1 11 11 00 00	01F0	32	ROL	1111 1100	FC
13	OR	Y xxx1 11 10 00 00	01E0	33	RCL	1111 0111	F7
14	JMP	Y 001x xx xx xx xx	0200				
15	JC	Y 010x xx xx xx xx	0400				
16	JNC	Y 011x xx xx xx xx	0600				
17	JZ	Y 100x xx xx xx xx	0800				
18	JNZ	Y 101x xx xx xx xx	0A00				
19	CALL	Y 110x xx xx xx xx	0C00				
20	RET	Y 111x xx xx xx xx	0E00				

The Table 1.15 includes other common instructions not presented earlier. The table presents conditional and unconditional branch instructions like JMP, JNC, JC, JZ, JNZ, CALL, RET etc. that are generally present in computing devices. Here JMP is an unconditional branch instruction that allows to change the order of instructions executed by the CPU. The JNC, JC, JZ and JNZ are conditional instructions that branch and change the order of instructions executed by the CPU when the condition is satisfied i.e. Carry flag – C or Zero flag – Z is set in JC and JZ and Carry flag – C or Zero flag – Z is reset in JNC and JNZ. The CALL and RET instructions are also unconditional instructions and are used with subroutines. The CALL instruction allows the branch transfer and execution subroutine, whereas RET is a return instruction that allows to transfer back to the main program from the subroutine. The specific instructions of PIC18 are presented in chapter 4.

The instructions 21 to 33 may be individual instructions or may be used as part of operations of instructions 1 to 20. The table 1.16 presents the hex code bit combination of ALU and shift/rotate operations for MOV and ADD instructions in combination with different shift/rotate operations.

Table 1.16: Example of instructions for combined Arithmetic, Logical and Shift/Rotate operations & corresponding Binary, HEX code bits.

S. No	OP Code	OP Code Binary - Combined SY-xxx9 87 65 43 21 S0-xABC DEFG		Hex Code
1	MOV No Shift	Y xxx0 11 11 11 00	0000 0000	00FC 00
2	MOV ASR	Y xxx0 11 11 11 00	1010 0000	00FC A0
3	MOV ASL	Y xxx0 11 11 11 00	1011 0000	00FC B0
4	MOV LSR	Y xxx0 11 11 11 00	1100 0000	00FC C0
5	MOV LSL	Y xxx0 11 11 11 00	1101 0000	00FC D0
6	MOV ROR	Y xxx0 11 11 11 00	1110 1000	00FC E8
7	MOV RCR	Y xxx0 11 11 11 00	1110 0110	00FC E6
8	MOV ROL	Y xxx0 11 11 11 00	1111 1100	00FC FC
9	MOV RCL	Y xxx0 11 11 11 00	1111 0111	00FC F7
10	ADD No Shift	Y xxx0 11 00 00 00	0000 0000	00C0 00
11	ADD ASR	Y xxx0 11 00 00 00	1010 0000	00C0 A0
12	ADD ASL	Y xxx0 11 00 00 00	1011 0000	00C0 B0
13	ADD LSR	Y xxx0 11 00 00 00	1100 0000	00C0 C0
14	ADD LSL	Y xxx0 11 00 00 00	1101 0000	00C0 D0
15	ADD ROR	Y xxx0 11 00 00 00	1110 1000	00C0 E8
16	ADD RCR	Y xxx0 11 00 00 00	1110 0110	00C0 E6
17	ADD ROL	Y xxx0 11 00 00 00	1111 1100	00C0 FC
18	ADD RCL	Y xxx0 11 00 00 00	1111 0111	00C0 F7

The HEX code format used is dependent on the actual hardware present. i.e. based on the Fig 1.26, Fig 1.27 & Fig 1.28. However, the complete HEX code using hardware structure of Fig 1.27 has not yet been included. For a 3 bus structure format, that may be using 3 registers – source 1,2 and destination as part of operand, the corresponding register select bits must also be included as part of instruction HEX code.

For a source 1 and source 2 as E, F registers; and destination as H register the binary select HEX code is xxx 1 11 10 0 101 i.e. 1E5H. The binary select bits for source 1 is QQQ (i.e. RSA210) corresponding to register E is 100. The binary select bits for source 2 is RRR (i.e. RSB210) corresponding to register F is 101. The binary select bits for destination is PPP (i.e. RD210) corresponding to register H is 111. The multi register bus structure for an ADD instruction without and with shift operation is presented in Table 1.17. The instruction Hex code format is shown in Fig 1.29.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
x	x	x	ALU/ Other Operation									Shift/Rotate			

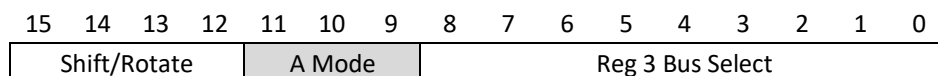


Fig 1.29: Instruction Hex Code Format

Table 1.17: Instruction example of ADD H, E, F including Arithmetic/Logical, Shift/Rotate and multi bus structure & corresponding Binary, HEX code bits. (Structure similar to ARM processor).

Instruction Mnemonic	OP Code Binary - Combined SY-xxx9 87 65 43 21 S0-xABC DEFG SZxxx-(RD210) (RSA210) (RSB210)		Hex Code
ADD H, E, F	Y xxx0 11 00 00 00	0000 0000 xxx 1 11 10 0 101	00C0 00 1E5
ADD H, E, ASL F	Y xxx0 11 00 00 00	1011 0000 xxx 1 11 10 0 101	00C0 B0 1E5

Here 00C0 forms the ADD operation, 1E5 forms the register select bits for H, E & F register selection and 00 for no shift and B0 for arithmetic shift left. The ADD H, E, ASL F instruction performs different operations. First the arithmetic left shift is performed on F register using shift rotate unit through the source 2 bus structure. It is then added to E register by the ALU through the source 1 bus structure. Finally, the result is stored into H register through the destination bus structure.

The demonstration presented in sec 1.8 does not present the possibilities for different addressing modes. However, a general computing device may also incorporate bits related to addressing mode as part of Hex code. The 3 registers are selected using 9 bits. Additionally, 3 bits are kept reserved making total 12 bits used for multi bus register selection. These reserved bits including some additional bits may be used for the addressing mode. The inclusion of addressing mode bits is left to the imagination of the readers as an assignment. Similarly, the hex code for different instructions with and without shift operations or addressing modes may be derived.

1.10 Optimization

Embedded systems are typically constrained devices that have an upper bound with aspects related to memory, speed, performance, efficiency etc. Hence they must be optimized to achieve the overall requirement and a compromise may be required on certain aspects. The optimization is achieved in software by the programmer or hardware by the computing device designer.

- Optimization in program using appropriate instructions and logic.
- Optimization in assemblers/compilers/synthesizers etc.
- A proper device selection (Microprocessor/Microcontroller) could yield better results. Ex: ARM processors use Thumb code to reduce program memory size etc. Similarly banking concept-based devices reduce program memory size.

- Additionally, certain devices have hardware designs to overcome data hazards, instructions hazard etc., include operand forwarding, branch prediction, branch target buffers, VLIW – Very Long Instruction Word, efficient pipelined structures etc. for achieving better results.

Table 1.18: Summary of various instructions & corresponding optimized HEX code bits.

S. No	OP Code	OP Code Binary SY-xxx9 87 65 43 21	Hex Code	Optimized Hex Code
1	MOV	Y xxx0 11 11 11 00	00FC	000
2	ADD	Y xxx0 11 00 00 00	00C0	010
3	ADC	Y xxx0 00 00 00 00	0000	020
4	SUB	Y xxx0 10 00 01 00	0084	030
5	SBB	Y xxx0 01 00 01 00	0044	040
6	INC	Y xxx0 10 00 11 00	008C	050
7	DEC	Y xxx0 11 00 10 00	00C8	060
8	NOT	Y xxx0 11 11 11 01	00FD	070
9	NEG	Y xxx0 10 00 11 01	008D	080
10	XOR	Y xxx0 11 11 00 00	00F0	090
11	XNOR	Y xxx0 11 10 00 00	00E0	0A0
12	AND	Y xxx1 11 11 00 00	01F0	0B0
13	OR	Y xxx1 11 10 00 00	01E0	0C0
14	JMP	Y 001x xx xx xx xx	0200	0D0
15	JC	Y 010x xx xx xx xx	0400	0E0
16	JNC	Y 011x xx xx xx xx	0600	0F0
17	JZ	Y 100x xx xx xx xx	0800	100
18	JNZ	Y 101x xx xx xx xx	0A00	110
19	CALL	Y 110x xx xx xx xx	0C00	120
20	RET	Y 111x xx xx xx xx	0E00	130
S. No	OP Code	OP Code Binary S0-xABC DEFG	Hex Code	Optimized Hex Code
21	No Shift	0000 0000	00	0
22	ASR	1010 0000	A0	1
23	ASRC	1010 0010	A2	2
24	ASL	1011 0000	B0	3
25	ASLC	1011 0011	B3	4
26	LSR	1100 0000	C0	5
27	LSRC	1100 0010	C2	6
28	LSL	1101 0000	D0	7
29	LSLC	1101 0011	D3	8
30	ROR	1110 1000	E8	9
31	RCR	1110 0110	E6	A
32	ROL	1111 1100	FC	B
33	RCL	1111 0111	F7	C

An example for design optimization is presented in Table 1.18 comparing with implementation of different operations presented in Table 1.15, Table 1.16, Table 1.17.

The optimization is achieved by assigning sequentially ordered numbers to the different instructions instead of just concatenating different select line bits.

Observe that the number of operations considered here are 20 in addition to the 13 shift and rotate operations. However typical operations exceed 20 and reach up to 100. Hence the higher three bits 21-23 are reserved in main ALU operations and 3 bits.

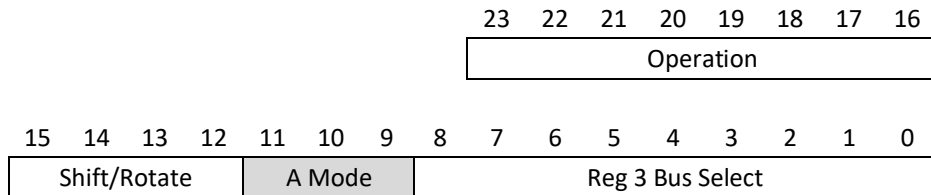


Fig 1.30: Optimized Instruction Hex Code Format

Table 1.19: Instruction example of ADD H, E, F including Arithmetic/Logical, Shift/Rotate and multi bus structure & corresponding optimized HEX code bits. (Structure like an ARM processor).

Instruction Mnemonic	Hex Code (32-bit)	Optimized Hex Code (24-bit)
ADD H, E, F	00C0 00 1E5	0C0 1E5
ADD H, E, ASL F	00C0 B0 1E5	0CB 1E5

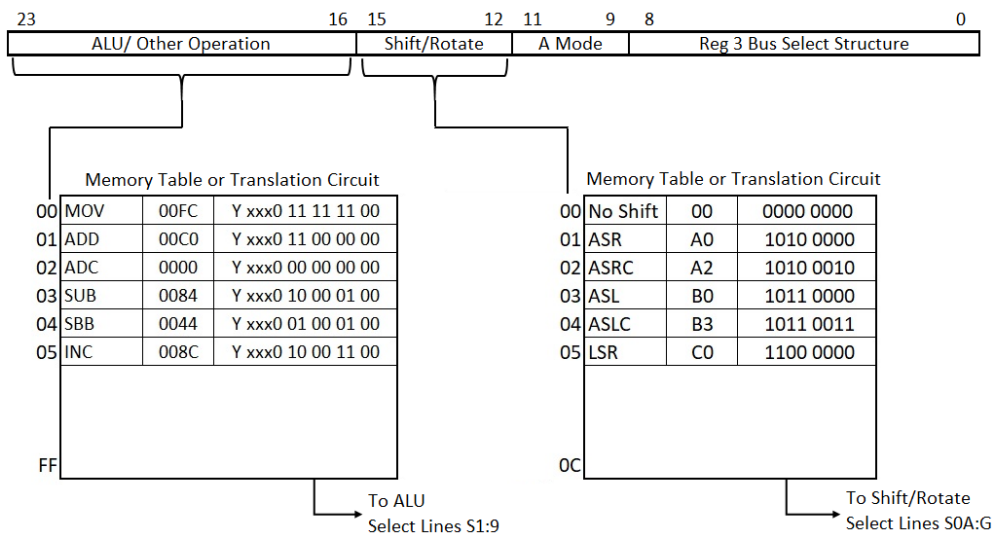


Fig 1.31: Optimized Instruction Hex Code Format

Here the higher 8 bits (bit 23:16) correspond to ALU operations, allowing a total of 256 instructions. The 4 bits (bit 15:11) correspond to shift/rotate operations. So, the

instruction operation codes take the values of 000 to FFCH where the lower digit varies from 0 to C which represent the shift/rotate operations. The higher digits 00 to FFH take the ALU and other operations.

The addressing modes are allocated 3 bits (bits 11:9) with 8 combinations. The final 9 bits (bits 8:0) correspond to 3 register selection. In case of 2 bus structure the 3 bits related to third bus structure may be used for additional addressing modes if present based on the design. The optimized instruction Hex code format is shown in Fig 1.30.

The 32-bit original code is now optimized and translated to a 24-bit code. So, each instruction is now stored with only 24 bits reducing the instruction size by 8 bits. So, a program of 100 instructions using non optimized code takes $32 \times 100 = 3200$ bits whereas the optimized code utilizes only $24 \times 100 = 2400$ bits. So higher the number of instructions, higher is the number of bits or bytes that are utilized for storing the program.

The above demonstration serves as an example for instruction code optimization, and it is achieved through a translation circuit as shown in Fig 1.31. This is just a conceptual example to demonstrate the concept of optimized code that is similar but not an exact translation of ARM and Thumb instruction format conversions.

1.11 Timing & Control Unit

The timing and control unit is the main control centre of the processor. It controls the different sub components in the processor by enabling or disabling the corresponding components in a precisely timed sequence for execution of the given instruction. The processor additionally includes a decoder unit which generates the decoded signals. The control unit uses this decoded information signals to generate the required sequence of sub operation signals for the given instruction. These sub operations are carried out in a precise timed order using a clocked sequence counter. The sequence of operations is presented later in this section along with hardware timing and control unit.

The timing and control unit structures are two types. The simplified block diagrams of the two types are presented in Fig 1.32 to demonstrate the differences between them. The two types of timing and control units are:

- 1) Hardwired Timing & Control
- 2) Microprogrammed Control

The hardwired timing and control unit is a type of unit in which all the internal components and operational signal lines for a given instruction are hard wired to form a fixed circuit. Since it is based on a hard-wired circuit, the circuit connections that are made are fixed permanently. Hence the internal operation signals & its sequence for a given instruction cannot be modified.

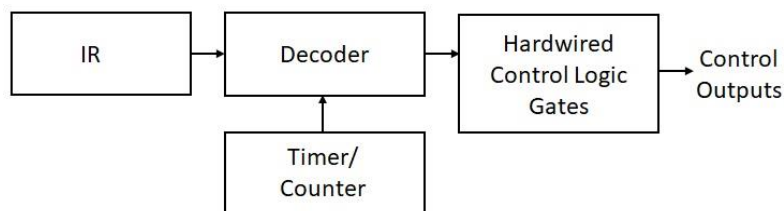


Fig 1.32a: Timing & Control Unit – Hardwired

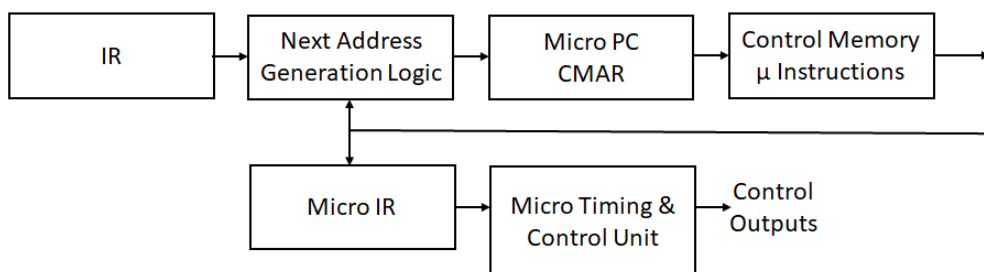


Fig 1.32b: Timing & Control Unit – Microprogrammed

The microprogrammed timing and control unit is a type of unit which divides the actual instruction into a series of microinstructions. This series of microinstructions are stored into a separate internal control memory. In addition to the control memory, it consists of a separate control memory address register that acts as a Micro Program Counter for fetching next micro instruction from the control memory. It also contains a Micro Instruction register for hold the micro instruction. The next address generation logic determines the address of next microinstruction to be fetched. A Micro timing and control unit performs the required operation sequence of a microinstruction.

Microprogrammed timing and control unit is different from a hardwired timing and control due to this addition of the internal control memory, Microinstructions, Micro PC and micro-IR. Since each instruction is composed of a set of microinstructions, the sequence of microinstructions for a given instruction could be changed at any time by just changing the sequence of microinstructions. This provides operational flexibility and reconfigurability, that is not available in a hardwired control.

Hardwired Timing and Control Unit:

A generalized structure of a hardwired timing and control unit is shown in Fig 1.33. The designs & optimization presented earlier in sections 1.8 and 1.9 are used to demonstrate the design of the Hardwired Timing and Control Unit. The hardwired unit consists of multiple decoders to decode the shift/rotate operations, ALU or other operations and timing.

The decoders generate a main control signal corresponding to the given /instruction and are named as DSR1:DSR15 for Shift/Rotate, DAOP1:DAOP255 for ALU Op and T0:T1n for the timing decoder. Only one output of a decoder is active at any instant. It is used by the

control logic gates to generate the required control output. The timing of the control outputs is enabled by the outputs of the SC Timing decoder. The sequence counter changes from 00000 to a M-bit value and correspondingly activates decoder outputs T0 to Tn for every instruction. So, a required control signal/operation is performed during a decoder timing output Tx where only one timing decoder is active at any given time.

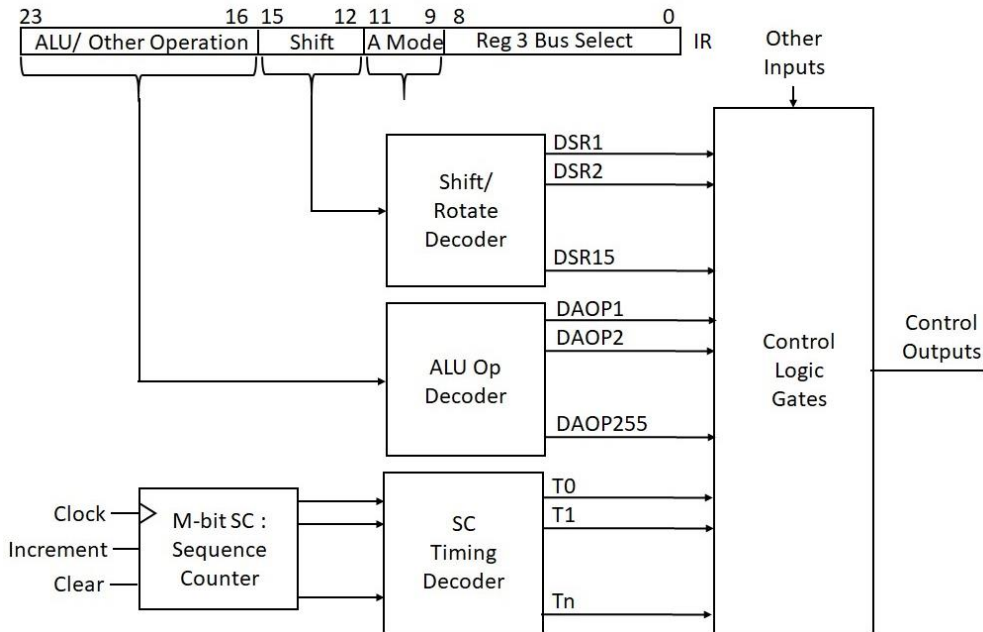


Fig 1.33: Timing & Control Unit - Hardwired

The operation sequence for instructions such as move, add and store have been presented in the section of operational concept of computer in section 1.7. These have been presented here step wise to demonstrate the operation of a Timing and Control Unit.

Common Operation sequence:

Load the PC with the starting address of the application program. So PC = 0x0000. (Only at program start).

1. Copy the contents of PC to MAR using internal control signals, i.e. internal IRD to PC and internal IWR to MAR. So MAR = PC = 0x0000. When timing decoder output T0 is activated, the IRD is generated to PC and IWR is generated to MAR.
2. Copy the contents of respective memory location (Hex Code) to MDR using the external control signals XRD to memory and internal write signal IWR to MDR. So MDR = [PC] = [MAR] = 0x07. When timing decoder output T1 is activated, the XRD is generated to memory and internal IWR is generated to MDR

3. Copy the contents of MDR to IR using internal control signals IRD to MDR and internal write signal IWR to IR. So $IR = MDR = 0x07$. When timing decoder output T2 is activated, the IRD is generated to MDR and an internal IWR is generated to IR.
4. Increment PC by 1 for every 1-byte access of memory. So $PC = PC + 1 = 0x0001$. When timing decoder output T3 is activated, the PC is incremented by 1.
5. Decode the value in IR register to help the CU determine the next sequence of steps specific to the respective instructions. When timing decoder output T4 is activated, the decoding of ALU Op and Shift/Rotate bit values are decoded by the respective decoders. Based on the hardware and number associated bits, this may require multiple timing decoder T states.

Instruction specific steps starts here: Steps for the MVI A, #05H specific instruction

6. Copy the contents of PC to MAR using internal control signals. i.e. internal IRD to PC and internal IWR to MAR. So, $MAR = PC = 0x0001$. When timing decoder output T5 is activated, the IRD is generated to PC and an internal IWR is generated to MAR.
7. Copy the contents of respective memory location to MDR using the external control signal XRD to memory and internal write signal IWR to MDR. So $MDR = [PC] = [MAR] = 0x05$. When timing decoder output T6 is activated, the XRD is generated to memory and an internal IWR is generated to MDR.
8. Increment PC by 1 for every 1-byte access of memory. So, $PC = PC + 1 = 0x0002$. When timing decoder output T7 is activated, the PC is incremented by 1.
9. Copy the contents of MDR to A register using internal control signals IRD to MDR and internal write signal IWR to A register. So, $A = MDR = 0x05$. When timing decoder output T8 is activated, the internal IRD is generated to MDR and an IWR is generated to A register.

So, the execution of the instruction MVI A, #05H requires a total of 9 steps. It can be assumed that each step is performed in 1 clock cycle. Hence this instruction requires 9 clock cycles to complete the execution. The sequence counter in the timing and control unit may be now set to work as a mod 9 counter to provide count of 0 to 8 during which the 9 steps of the MVI A, #05H instruction is completed. i.e. when the counter value is 0 the step 1 is executed, for counter value of 1 the step 2 is executed and so on when the counter value is 8 the step 9 is executed and in next clock cycle the sequence is reset to 0. Based on the hardware setup certain steps may require additional T states and the later steps may be shifted by the respective number to be completed in given states.

Instruction specific steps starts here: Steps for the ADD A, B specific instruction

For an instruction such as the addition, steps 8 & 9 are not required and step 6 and 7 of MVI A, #05H instruction would be replaced as follows.

6. Read the register A and register B using internal read control signals IRD and make it available for ALU. When timing decoder output T5 is activated, the IRD is generated to both A and B registers to make data available to ALU.
7. Enable the ALU using internal control signal and configure to perform addition operation. The configuring of ALU has been presented earlier in the ALU section 1.9. When timing decoder output T6 is activated, the ALU may be configured and made perform the given ALU operation.
8. After addition is performed in the ALU, the result 08H is stored in register A (or C) by an internal control write signal IWR to A (or C) register. When timing decoder output T7 is activated, the IWR is generated to A (or C) register to store the result.

Hence this instruction requires 8 clock cycles to complete the execution. The sequence counter in the timing and control unit may be now set to work as a mod 8 counter to provide count of 0 to 7 during which the 8 steps of the ADD A, B instruction are completed.

Instruction specific steps starts here: Steps for the STO A, 10H specific instruction

For an instruction such as the STO A, 10H, the step 6 to 9 of MVI A, #05H instruction would be replaced as follows.

Memory Read Operation:

6. Copy the contents of PC to MAR using internal control signals. i.e. internal IRD to PC and internal IWR to MAR during T5. So, $MAR = PC = 0x0006$.
7. Copy the contents of respective memory location to MDR using the external control signal XRD to memory and internal write signal IWR to MDR during T6. So $MDR = [PC] = [MAR] = 0x10$.
8. Increment PC by 1 for every 1-byte access of memory. So, $PC = PC + 1 = 0x0007$ during T7.
9. Copy the contents of MDR to MAR register using internal control signals IRD to MDR and internal write signal IWR to MAR register during T8. So $MAR = MDR = 0x0010$

Memory Write Operation:

10. Further the A (or C) register is enabled through an internal read signal IRD which places the data-on-data bus. It is then copied to MDR register by enabled through an internal write signal IWR to store the A register value 08H in MDR register during T9. So $MDR = C = 0x08$.
11. The addition result present in MDR is written to memory location 0x0010H (pointed by MAR) when the control unit generates an external write signal XWR to memory and an internal read signal IRD to the MDR register during T10.

Hence this instruction requires 11 clock cycles to complete the execution. The sequence counter in the timing and control unit may be now set to work as a mod 11 counter to provide count of 0 to 10 during which the 11 steps of the STO A, 10H instruction is completed.

Microprogrammed Timing and Control Unit:

A generalized structure of a microprogrammed timing and control unit is shown in Fig 1.34. As presented earlier a microprogrammed control includes various additional components w.r.t a hardwired control.

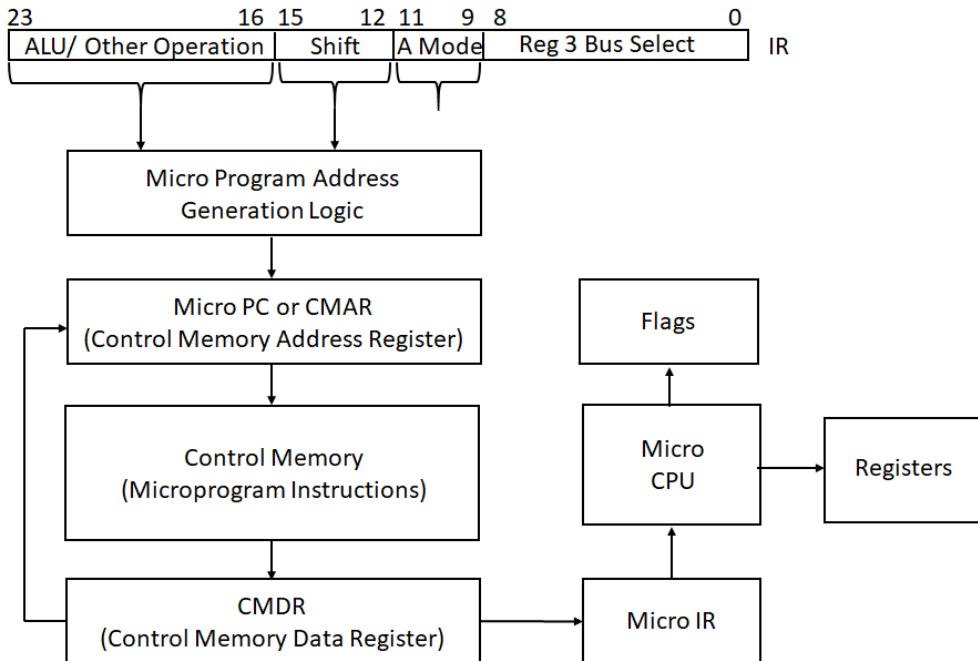


Fig 1.34: Timing & Control Unit – Microprogrammed

In a microprogrammed unit every instruction is composed of a group of microinstructions. These microinstructions are stored in a local control memory that is incorporated specifically for the microprogrammed timing and control unit. A Micro PC or a control memory address register - CMAR is included to perform the function similar to a normal PC. However, the micro PC refers the control memory consisting of the microinstructions. A control memory data register is used to temporarily hold the microinstruction code before it is transferred to the next register. Similarly, a micro IR is included which performs similar function of an IR. However, the Micro IR here stores the microinstruction code. Each microinstruction may correspond to a step in a normal hardwired control. To execute a normal instruction, a set of microinstructions are executed in a Micro CPU in a defined order. This may later be changed based on the requirement by just changing the microinstructions in the control memory.

Every instruction is mapped to a specific address in the control memory (similar to an interrupt vector address for interrupts). This corresponds to the address of first microinstruction in control memory. A mapping logic is used to map the instruction to the

microinstructions starting address. So whenever an instruction is to be executed, the execution is transferred to microinstructions address in the control memory. The microprogram address generation logic is used to determine the address of the next microinstruction that must be executed.

The Micro IR is shown separately in the Fig 1.34 only to highlight its existence and that its function is similar to that of IR. The Micro CPU may vary based on the design. It may be designed similar to a normal CPU to execute microinstructions at a lower level or represent a micro control unit with control logic gates that generates the required internal signals for the respective microinstruction. The registers and flags are integral to the main control unit and hence are presented outside the Micro CPU. The Micro CPU internally may contain its own registers based on the design.

1.12 Pipelining

A microcontroller performs its operations as a sequence of steps that are completed in different hardware units. The sequence of steps is to be executed in corresponding units based on timing cycle derived from the crystal oscillator clock. To increase the efficiency, hardware and its units is organized as sequential units so that the units process its tasks and work independent of one another i.e. work in parallel.

Pipelining

The process of parallelising or overlapping the execution of different hardware units to perform the required tasks is known as pipelining.

The three common pipeline units are Fetch, Decode and execution units. The Fetch unit perform the process of getting the instruction code from program memory to a Queue or instruction register. The decoding unit perform the process of determining the required operations for the corresponding instruction. The Execution unit performs the execution of the required operations.

The Fig 1.35 shows the timing diagram for a 3-Stage pipeline where stage 1 involves Fetch unit while decode unit form the stage 2 and the execution for the stage 3. The fetching of instruction 1 is carried out by stage 1 unit during the first clock cycle C0. During the second clock cycle C1, stage 1 unit performs fetching of second instruction. The stage 2 unit operates in parallel to stage 1 and performs the decode of first instruction. During the third cycle C2 the stage 1 unit performs fetching of third instruction. The stage 3 unit operates in parallel to stage 1 and stage 2 and performs the execution of first instruction. During this time C2, the stage 1 fetches the third instruction, stage 2 decodes the second instruction and stage 3 executes the first instruction. This completes the execution of first instruction and henceforth an instruction would be executed for every C cycle. i.e. the operation performed

in different stages during C2 cycle is repeated for the next C cycles till the completion of execution of all the instructions.

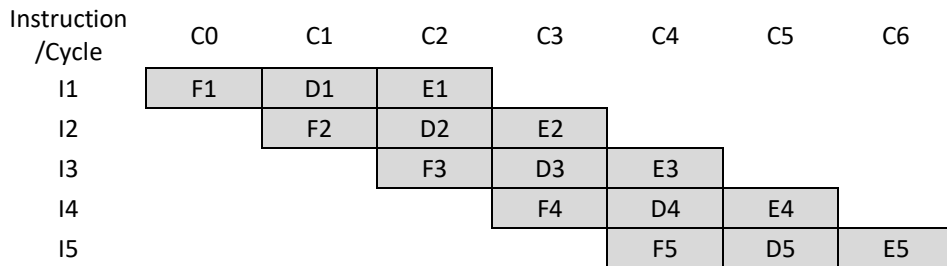


Fig 1.35: Pipeline diagram for execution of five instructions with 3 state pipeline.

Another common setup uses a four stage pipeline. The 4 stage pipeline utilized Fetch, Decode, Execute and a Memory Write stage. The result generated after the execution of instruction is commonly stored back into memory. Hence some devices employ this as the fourth stage.

The number of pipeline stages vary based on the design. The Intel 8086 uses a two stage pipeline. PIC18 also uses a 2 stage pipeline. The Intel 80286, ARM7 TDMI and ARM Cortex M3 uses a 3 stage pipeline. The Intel 80486, ARM9 TDMI uses 5 stage pipeline, Intel P5, ARM10 TDMI uses 6 stage pipeline and ARM11 TDMI uses 8 stage pipeline. Similarly, devices exist with pipeline stages of the order 20 and more.

Cycles – Clock Cycle, State, Machine Cycle, Instruction Cycle

Each device operates on a specific clock frequency. This the smallest increment that can be detected by the device. The time take for completion of one cycle is considered as a clock cycle time or clock period, denoted as CC.

An operation represents one memory read or one memory write etc. Any given operation is composed of several uniformly divided sub operations. These sub operations may be performed by a corresponding hardware unit. The time or number of clock cycles required for completion of one sub operation may be termed as one state. So a State consists of multiple clock cycles and is denoted as S. So the State $S = n * CC$, where n represents number of cycles required.

Similarly, a machine cycle is a combination of these sub operations and represents the number of cycles required for completion of an operation. This operation may involve multiple states. Hence a machine cycle consists of multiple states. So Machine Cycle $M = r * S$, where r represents number of states.

An instruction is generally a combination of many operations. Hence an instruction cycle consists of multiple machine cycles. So Instruction cycle $I = t * M$, where t represents the number of operations in an instruction.

To summarize the above, the clock cycle time is the smallest incremental time step of the device. A state is the time required for completion of a hardware unit or the time required to perform a sub operation. Machine cycle time is the time required or the number of clock cycles required to perform an operation. An instruction is a combination of one or more operations. Hence the instruction cycle time is the number of machine cycles required for execute an instruction.